From Token-Time to Episode-Time: A Semantic Runtime and Dissipative Control Framework for Stable Attractor-Based LLM Systems

Natural Semantic Clocks, Sub-Attractor Coordination, and Boundary-Timed Intervention for Production LLM Agents

0. Reader Contract and Engineering Scope

This paper proposes a practical engineering shift in how we model higher-order LLM behavior. Its central claim is simple: token count and wall-clock time remain valid low-level clocks, but they are often the wrong primary clocks for analyzing multi-step semantic coordination. This claim was first developed in **"Coordination-Episode Tick: The Natural Time Variable for Attractor-Based LLM and AGI Dynamics"** and later extended toward runtime form in **"從心跳到 LLM 韻律單元探究."**

The scope of this paper is deliberately narrow and operational. It is not a theory of consciousness. It is not a claim that all LLM behavior must be modeled with attractors. It is not an attempt to replace standard autoregressive analysis. Instead, it asks a concrete systems question:

What is the right engineering clock for describing meaningful progress in systems that retrieve, branch, deliberate, call tools, revise, and finally export a usable artifact?

The answer proposed here is the **coordination episode**. A coordination episode is a variable-duration semantic unit defined not by fixed seconds or fixed token counts, but by the completion of a bounded coordination process. Such a process begins when a meaningful trigger activates one or more local reasoning structures, and it ends when a locally stable, transferable output has been formed. This is the exact conceptual bridge that turns a time-theory paper into a runtime paper.

In this paper, "stable" does not mean globally correct, metaphysically deep, or immune to error. It means only that a local process has reached a sufficiently coherent closure to export an artifact into the next stage of reasoning. A system can still fail by stabilizing too early, looping inside a bad local basin, or exporting a fragile intermediate result. The runtime view therefore needs both a notion of closure and a notion of fragile or pathological closure. This engineering stance is already present in **"從心跳到 LLM 韻律單元探究,"** which distinguishes robust closure, fragile closure, loop capture, non-convergence, asymmetry block, and pending states.

This paper is written for engineers building:

- tool-using LLM agents,
- long-context reasoning services,
- structured-output systems,
- planner–executor loops,
- multi-module runtimes,
- and future multi-agent coordination systems.

For such systems, the practical question is no longer only "what token came next?" but also:

- what local semantic process is active now,
- why it was triggered,
- whether it is converging,
- whether it is fragile,
- and when intervention is most likely to help.

The engineering objective is therefore not just better description, but better control. In **"Dissipative Lagrangian Decoding: Event-Triggered Short-Horizon Control for Stable, On-Task Large Language Models,"** the control problem is framed at the per-token level: reduce drift, format breakage, and erratic tool use without retraining, using a bounded inference-time controller with trust-region guards. That paper provides the control vocabulary this paper will reuse and lift to the episode level.

At the most compressed level, the paper starts from the familiar micro-update picture:

$$x_{n+1} = F(x_n) \quad (0.1)$$

For decoder-only generation, this is the correct local view. But for higher-order semantic coordination, we propose the complementary episode-level view:

$$S_{k+1} = G(S_k, \Pi_k, \Omega_k) \quad (0.2)$$

where:

- k indexes completed coordination episodes,
- S_k is the episode-level semantic/runtime state before episode k,
- Π_k is the active coordination program during the episode,
- Ω_k is the set of observations, retrieved memory, tool outputs, constraints, and external signals encountered during the episode.

The reader contract is therefore straightforward:

1. Keep token-time for microphysics.
2. Introduce episode-time for semantic coordination.
3. Build runtime objects around episode structure.
4. Add dissipative control where fragility, drift, or boundary-risk appears.

That is the paper's scope.

---

1. Why Token-Time Is the Wrong Clock for Higher-Order LLM Behavior

Any theory of system dynamics must choose a time variable. In today's LLM practice, three clocks dominate by default:

- **token-time**, where each next-token step counts as one update;
- **wall-clock time**, measured in milliseconds or latency intervals;
- **simple event counts**, such as tool calls, turns, retries, or loop iterations.

All three are useful. None should be discarded. But for higher-order semantic coordination, none is obviously the natural clock.

Token-time is the obvious starting point because base LLMs are built autoregressively. At the micro-level, the picture is exact enough:

$$h_{n+1} = T(h_n, x_n) \quad (1.1)$$

This is the right language for studying local mechanisms such as attention flow, hidden-state evolution, layerwise transport, and next-token selection. If the object of study is local decoding behavior, token index n is the right coordinate.

The difficulty appears when token count is promoted from a microphysical index to a semantic clock. A system can emit many tokens while making no meaningful progress. It can also undergo a major semantic reorganization in a short and compact burst. A short retrieval result may completely change the reasoning trajectory. A long self-explanation may add surface elaboration without changing the operative task state. In other words, token progression and semantic progression are not reliably synchronized. This mismatch is stated explicitly in both **"Coordination-Episode Tick: The Natural Time Variable for Attractor-Based LLM and AGI Dynamics"** and **"從心跳到 LLM 韻律單元探究."**

Wall-clock time is even less satisfactory as a semantic clock. Elapsed seconds depend on hardware, batching, queueing, network latency, tool APIs, and runtime architecture. Two semantically identical reasoning episodes may take different wall-clock durations, while two semantically different episodes may consume similar latency. Wall-clock time is indispensable for serving and SRE work, but it is usually not the right variable for semantic phase analysis.

Simple event counts improve on raw clock time, but only partially. Counting tool calls, branch points, retries, or turns is more structured than counting seconds, yet it still assumes that the counted event matches the real semantic boundary. Often it does not. A single reasoning episode may involve several tool calls but still be one coherent semantic push. Conversely, a single visible turn may hide multiple internal sub-processes. The problem is therefore not only which event

to count, but whether the event count has been anchored to the correct unit of semantic change.

This can be compressed into one principle:

A good time axis must align with the natural granularity of the state changes we want to explain.

If the state changes of interest occur when local conflicts are resolved, candidate interpretations are arbitrated, evidence is fused, or a transferable artifact is formed, then a much finer or much coarser clock will distort the observed geometry of the process. A trajectory that looks noisy in token-space may become structured in episode-space. A process that looks continuous at output level may decompose into several distinct semantic basins when indexed by coordination episodes. This is exactly why time-variable choice is not cosmetic in an attractor-based analysis.

The critique can therefore be stated directly:

$n \neq$ natural time for high-order reasoning (1.2)

$t \neq$ natural time for semantic coordination (1.3)

These equations do not say token index n or wall-clock t are false. They say only that, for many higher-order reasoning tasks, they are misaligned with the semantic structure one is trying to model. This claim is central in **"Coordination-Episode Tick: The Natural Time Variable for Attractor-Based LLM and AGI Dynamics."**

A simple example makes the issue concrete. Suppose the system must answer a difficult binary question. The final output is only "True" or "False." But internally the system may have to:

- identify the claim type,
- clarify an ambiguous criterion,
- generate a rival interpretation,
- arbitrate between two local basins,
- and only then compress the resolved structure into a one-bit verdict.

This kind of runtime decomposition is explicitly illustrated in the worked example of **"Coordination-Episode Tick: The Natural Time Variable for Attractor-Based LLM and AGI Dynamics."** There, the meaningful state changes occur when structured artifacts are formed, not when arbitrary token boundaries are crossed.

The same mismatch becomes stronger in tool-using and multi-agent systems. A planning loop may require a variable-duration subgoal cycle before any meaningful state transition has occurred. A tool call may or may not complete a semantic episode. A multi-agent exchange may only matter semantically when a coordinated closure event is reached. The benefit of episode-time is therefore expected to increase with coordination complexity, a prediction already made explicitly in **"Coordination-Episode Tick: The Natural Time Variable for Attractor-Based LLM and AGI Dynamics."**

So the problem is not that today's clocks are useless. The problem is that they are often clocks for the wrong layer. Token-time is the microphysical clock of generation. Episode-time is proposed here as the more natural engineering clock for semantic coordination.

---

2. Coordination Episodes as the Natural Semantic Time Variable

We now define the paper's central object.

A **coordination episode** is the smallest variable-duration semantic unit such that:

(i) a meaningful trigger initiates one or more local processes,
(ii) those processes interact under bounded tensions and constraints,
(iii) a local convergence condition is reached, and
(iv) a transferable output is produced. (2.1)

This definition is the operational core of the earlier paper **"Coordination-Episode Tick: The Natural Time Variable for Attractor-Based LLM and AGI Dynamics."** It is also the key step that lets engineers reinterpret attractor language as runtime language.

The important point is that an episode is defined by **closure**, not by duration. It may be short or long. It may contain few or many micro-steps. It may involve no tool call, one tool call, or many. What makes it one episode is that it forms one bounded semantic push that changes what the system can now responsibly export, suppress, or act on. This is why the episode index is a candidate natural semantic clock.

The formal shift is:

$x_{n+1} = F(x_n)$ (2.2)

$S_{k+1} = G(S_k, \Pi_k, \Omega_k)$ (2.3)

where `n` indexes low-level micro-updates, while `k` indexes completed coordination episodes. The engineering claim is not that equation (2.2) is wrong, but that equation (2.3) often better captures the state transitions relevant to reasoning, orchestration, and intervention. This distinction is stated directly in **“從心跳到 LLM 韻律單元探究.”**

To make this computable, the paper adopts a minimal runtime notion of local process. The smallest reusable local unit is the **semantic cell**:

$C = (I, En, Ex, X_{in}, X_{out}, T, \Sigma, F)$ (2.4)

where:

- `I` = intent,
- `En` = entry conditions,
- `Ex` = exit criteria,
- `X_in` = required inputs,
- `X_out` = expected outputs,
- `T` = referenced tensions,
- `Σ` = observable signals,
- `F` = failure markers.

This semantic-cell schema is important because it prevents the framework from remaining metaphorical. Once a local reasoning unit has entry conditions, exit criteria, signals, and failure markers, it can be logged, instrumented, scheduled, and evaluated. The semantic cell is therefore the local building block of episode-time.

At runtime, a coordination episode is not just “one cell ran.” It is a bounded coordination process over multiple candidate and active cells. The minimal operations are:

- **Trigger** — which cells become relevant?
- **Routing** — which candidates are actually activated?
- **Convergence** — which active cells have stabilized enough?
- **Composition** — how do locally converged outputs combine into a larger reasoning state?

These four operations define the minimal semantic runtime of attractor-based reasoning. They are the runtime translation of the earlier attractor language.

A simple formal sketch is:

$a_i(k) = H_i(S_k, T_k, \Omega_k)$ (2.5)

$i^* = \operatorname{argmax}_i a_i(k)$ (2.6)

$q_i(k) \geq \theta_i^{conv}$ (2.7)

$Y_{k+1} = Comp(\{X_{out}^{(i)}\}_{i \in A_k^{conv}}, R_k, T_k)$ (2.8)

Interpretation:

- `a_i(k)` scores which local cell should activate,
- `i*` selects or prioritizes activation,
- `q_i(k)` measures local convergence,
- `Y_(k+1)` is the composed artifact exported by the episode.

This general runtime grammar is already outlined in **“從心跳到 LLM 韻律單元探究.”**

A coordination episode therefore ends not when a timer expires, but when an episode-level completion condition is met. The simplest completion indicator is:

$\chi_k = 1$ if episode k has reached transferable closure; 0 otherwise (2.9)

But closure alone is not enough. Some episodes finish badly. Some stabilize inside the wrong basin. Some appear complete but are structurally weak. For this reason, the framework also distinguishes outcome classes such as:

- COLLAPSED,
- COLLAPSED_BUT_FRAGILE,
- ATTRACTOR_LOOP,
- NOT_CONVERGED,
- ASYMMETRY_BLOCK,
- PENDING.

This outcome taxonomy matters because it prevents the new time variable from collapsing back into crude event counting. If semantic time is defined by coordination closure, then the runtime must also classify what kind of closure occurred. Otherwise every terminated process would count equally, and the model would lose its explanatory value. That argument is made explicitly in **“從心跳到 LLM 韻律單元探究.”**

Finally, this definition of episode-time generates testable engineering predictions. If coordination episodes are the more natural semantic units, then:

- episode-indexed traces should reveal cleaner structure than token-indexed traces,
- episode metrics should detect failure earlier than token metrics,
- and interventions should work better near episode boundaries than at arbitrary token positions. These predictions are made directly in **“Coordination-Episode Tick: The Natural Time Variable for Attractor-Based LLM and AGI Dynamics.”**

So Section 2 leaves us with one practical thesis:

For higher-order LLM systems, the natural engineering clock is often not the next token, but the next completed coordination episode.

When that is true, semantic-time stops being a philosophical metaphor and becomes a runtime design principle.

3. From Attractor Metaphor to Semantic Runtime

Up to this point, the paper has argued that higher-order LLM behavior is often better indexed by completed coordination episodes than by raw token count. That claim is useful, but by itself it is still only half an engineering theory. To become operational, the language of “attractors” must be translated into runtime objects that can be inspected, scheduled, monitored, and controlled. This translation was partially sketched in **“Coordination-Episode Tick: The Natural Time Variable for Attractor-Based LLM and AGI Dynamics”** and made more explicit in **“從心跳到 LLM 韻律單元探究.”**

In many current discussions, “attractor” is used as an evocative metaphor. One says a system “falls into a basin,” “stabilizes on an interpretation,” or “gets trapped in a loop.” These descriptions are intuitively useful, but they remain too vague for runtime engineering. A production system cannot monitor a metaphor. It needs objects, states, transitions, signals, and control points.

The shift proposed here is simple:

A local attractor is reinterpreted as a bounded semantic process with recognizable entry conditions, local dynamics, and exportable closure.

Once stated this way, the engineering meaning becomes much clearer. A retrieval check, a contradiction-resolution pass, a tool-selection deliberation, a schema repair attempt, or a branch arbitration step can each be treated as local semantic processes. These are not necessarily independent modules in the software stack, but they are meaningful units in the runtime analysis. This is exactly the direction pointed to in **“從 LLM「傾蓋如故」和「白首如新」的差異對待 探討構造 AGI Attactor 所必須的 Heatbeat 時間本質,”** where the research agenda is framed in terms of trigger, routing, local convergence, composition, arbitration, scale transition, and failure mode rather than in terms of one monolithic global attractor.

This leads to an important representational shift. Instead of assuming that reasoning is one opaque hidden trajectory, we model it as an ecology of local semantic units. Some are candidates but never activate. Some activate briefly and fail. Some converge and export useful artifacts. Some compete with one another. Some cooperate and compose into a larger

state. This ecological view is stated directly in **"The Natural Time Variable for Attractor-Based LLM and AGI Dynamics,"** which notes that routing may involve winner-take-all selection, thresholded multi-activation, inhibitory gating, cooperative activation, and priority overrides.

At this point, attractor vocabulary can be rewritten into runtime vocabulary:

- a **basin** becomes a locally stable semantic mode,
- a **rival basin** becomes a competing candidate interpretation or branch,
- a **local minimum** becomes a provisional closure,
- a **bad basin** becomes a pathological closure such as loop capture or false certainty,
- a **basin transition** becomes a routed shift in active process structure,
- and a **higher-order attractor** becomes a composed state built from several local closures.

The key engineering point is that the paper does not require a metaphysical proof that the hidden state space literally contains mathematically exact attractors in a strict dynamical-systems sense. That would be a much heavier claim and is unnecessary here. What is needed is a runtime abstraction that behaves as if local semantic processes can stabilize, compete, fail, and compose in structured ways. This weaker stance is enough to guide instrumentation and intervention.

We therefore introduce the idea of a **semantic runtime**. A semantic runtime is the structured layer that tracks which local semantic processes are relevant, which become active, whether they are converging, what they export, and how their outputs reshape the next episode-level state. This is not merely another logging vocabulary. It is a proposed control surface for higher-order LLM systems. The earlier documents already point toward this explicitly: **"從心跳到 LLM 韻律單元探究"** describes four core runtime operations, while **"The Natural Time Variable for Attractor-Based LLM and AGI Dynamics"** develops them into a minimal engine of semantic tick runtime.

A useful first abstraction is to say that a coordination episode does not directly update only "the model." It updates a structured semantic configuration. One may write the episode-level state abstractly as:

S_k = (Z_k, A_k, T_k, M_k, R_k, Y_k) (3.1)

where:

- `Z_k` is a proxy for the latent semantic configuration,
- `A_k` is the set of active local processes,
- `T_k` is the current tension vector,
- `M_k` is memory and retrieved context,
- `R_k` is routing and arbitration state,
- `Y_k` is the set of currently exported artifacts.

This equation is not yet the whole runtime. It only says that an episode-level state must be composite. That composite view matters because most practical failures in advanced LLM systems are not best described as "the next token was wrong." They are better described as failures of activation, failures of selection, failures of local closure, or failures of higher-order assembly. **"The Natural Time Variable for Attractor-Based LLM and AGI Dynamics"** makes this especially clear by classifying failure separately for trigger, routing, convergence, and composition.

This section therefore performs one conceptual conversion:

Before this section, "attractor" could still be read as a metaphor for semantic stability.
After this section, it becomes a proposal for how to segment and manage higher-order reasoning as a runtime of bounded semantic units.

That conversion is what makes the rest of the paper possible.

---

4. The Minimal Runtime Unit: Semantic Cells

If a semantic runtime is to be engineered, it must have a smallest reusable local unit. That unit cannot simply be "a token," because the entire argument of the paper is that token-time is often too fine-grained for higher-order coordination. It also cannot simply be "a full prompt-response cycle," because that is often too coarse and hides multiple internally distinct sub-processes. The right unit must sit in between: small enough to be composable, but rich enough to carry semantic entry, exit, and monitoring structure.

We call this unit a **semantic cell**.

The semantic cell was already sketched in the appendices of **"The Natural Time Variable for Attractor-Based LLM and AGI Dynamics"** and expanded in **"從心跳到 LLM 韻律單元探究."** In both documents, the key move is the same: instead of treating a local reasoning act as an invisible blob, define a minimal object with intent, boundary conditions, signals, and failure markers.

The minimal semantic cell is:

$C_i = (I_i, En_i, Ex_i, X_{in}_i, X_{out}_i, T_i, \Sigma_i, F_i)$ (4.1)

where:

- I_i = local intent,
- En_i = entry conditions,
- Ex_i = exit criteria,
- X_{in}_i = required inputs,
- X_{out}_i = expected outputs,
- T_i = referenced tensions,
- Σ_i = observable signals,
- F_i = failure markers.

This definition is minimal but already useful. It tells us that a local semantic process is not identified by "what it is called" but by what conditions activate it, what signals it emits while active, what closure it tries to reach, and what kind of artifact it is expected to export.

A semantic cell may correspond to many concrete activities:

- a retrieval cell,
- a contradiction-check cell,
- a branch arbitration cell,
- a tool-selection cell,
- a schema-validation cell,
- a summarization cell,
- a local planning cell,
- or a formatting-repair cell.

The abstraction is deliberately role-based rather than implementation-bound. The same underlying model call may instantiate several semantic cells in sequence. Conversely, several software modules may jointly implement one semantic cell. What matters is that the runtime can recognize the cell as a meaningful unit of coordination.

A semantic cell is therefore more structured than a hidden-state fragment and less global than an entire reasoning trace. This is precisely why it is useful for engineers. It becomes possible to ask:

- Why was this cell activated?
- Did it receive the inputs it needed?
- Which tension axes was it balancing?
- Did it export an artifact or stall?
- Did it terminate cleanly or in a fragile way?

These are already the kinds of observability questions advocated in **"The Natural Time Variable for Attractor-Based LLM and AGI Dynamics,"** where the proposed runtime dashboard is asked to show active cells, current tensions, convergence score, contradiction residue, loop warnings, expected artifacts, episode state, and estimated fragility.

To make the cell operational, we also need a finite status vocabulary. The earlier work proposes the following local cell

statuses:

cell_status_i(k) ∈ {inactive, candidate, active, converged, fragile, looped, blocked} (4.2)

This matters because a runtime must distinguish between a cell that merely exists in the library, one that has become relevant, one that is actively consuming resources, and one that has locally stabilized or failed. This status structure is stated explicitly in **"The Natural Time Variable for Attractor-Based LLM and AGI Dynamics."**

The semantic cell also interacts with a structured tension model. A tension is not just "difficulty." It is a semantic axis along which local processes are being pulled, balanced, or forced to resolve trade-offs. The earlier runtime appendices define a tension object as:

Tension_j = (id_j, axis_j, weight_j, threshold_j, signal_j) (4.3)

with a runtime episode carrying a full evaluated tension vector:

T_k = (τ_1(k), τ_2(k), ..., τ_m(k)) (4.4)

This is not a cosmetic addition. Tensions are what connect cells to the larger semantic field. A retrieval cell may be pulled between recall and precision. A final answer cell may be balancing concision against justification. A tool router may be balancing latency against verification. Triggering, routing, convergence, and fragility all depend on these tensions. This object structure is named directly in **"The Natural Time Variable for Attractor-Based LLM and AGI Dynamics"** and **"從心跳到 LLM 韻律單元探究."**

Finally, a semantic cell is meaningful only because it can export an artifact. The earlier runtime schema defines artifacts as transferable outputs of local or global closure:

Artifact_r = (id_r, type_r, source_r, payload_r, quality_r, transferable_r) (4.5)

This is where the semantic cell becomes indispensable for engineering. Without artifacts, the runtime only knows that "something happened." With artifacts, it can track what changed: a candidate answer, an evidence bundle, a selected branch, a tool call request, a conflict-resolved local state, a repaired JSON segment, a summary object, or a routing decision. Again, the artifact object is explicitly defined in **"The Natural Time Variable for Attractor-Based LLM and AGI Dynamics."**

We can now state the practical reason this section matters:

A semantic runtime cannot be built from time variables alone.
It needs local units with entry logic, exit logic, signals, failures, and artifacts.
The semantic cell is that unit.

Once semantic cells exist, higher-order reasoning can be segmented into reusable, monitorable, and eventually controllable acts. This is what turns "episode-time" from a descriptive clock into an engineering design principle.

---

5. Trigger, Routing, Local Convergence, and Composition

With the semantic cell in place, the runtime can now be expressed through four primitive operations. These were named directly in **"從心跳到 LLM 韻律單元探究"** as the four core runtime mechanics and developed in more detail in **"The Natural Time Variable for Attractor-Based LLM and AGI Dynamics."**

The four primitives are:

1. **Trigger** — which semantic cells become relevant?
2. **Routing** — which candidate cells are actually activated?
3. **Local convergence** — when is a local process stable enough to export?
4. **Composition** — how do locally converged outputs reshape the next larger state?

Taken together, these four operations define the minimal engine of an episode-time semantic runtime.

5.1 Trigger

Triggering is the first gate. It answers the question: given the current episode-level state, which local cells become relevant enough to consider?

A trigger should not be treated as a purely symbolic rule or as an arbitrary software callback. In the runtime view, a trigger depends on the current semantic configuration, current tensions, and available observations. The earlier work gives the generic trigger score:

$a_i(k) = H_i(S_k, T_k, \Omega_k)$ (5.1)

where:

- `S_k` is the current episode-level state,
- `T_k` is the current tension vector,
- `Ω_k` is the incoming observation set,
- `a_i(k)` is the relevance score for cell `i`.

A cell may be triggered because a contradiction was detected, because a specific tool became relevant, because evidence is missing, because an output structure is incomplete, or because a macro-goal requires a particular local process. **“從 LLM「傾蓋如故」和「白首如新」的差異對待 探討構造 AGI Attactor 所必須的 Heatbeat 時間本質”** names this as the first real research mechanism in an attractor-based LLM theory and explicitly notes the analogy to conditional computation and MoE gating, while arguing that the needed shift is from token-to-expert routing toward sub-attractor-episode triggering.

The important engineering point is that triggering is not yet execution. It only creates a candidate set:

$A_k^{cand} = \{i : a_i(k) \geq \theta_i^{trig}\}$ (5.2)

The runtime should therefore distinguish between cells that are merely available in the library, cells that are relevant candidates now, and cells that actually receive resources.

5.2 Routing

Once candidate cells exist, the runtime must decide which of them become active. This is routing.

In the simplest case, routing is a winner-take-all decision:

$i^* = \arg\max_i a_i(k)$ (5.3)

But practical systems often need richer routing policies. **“The Natural Time Variable for Attractor-Based LLM and AGI Dynamics”** explicitly lists thresholded multi-activation, staged activation, inhibitory gating, cooperative activation, and priority overrides as possible routing mechanisms. It also introduces a more general routing score:

$r_i(k) = R_i(a_i(k), comp_i(k), pri_i(k), cost_i(k))$ (5.4)

followed by a routed active set:

$A_k = Route(S_k, T_k, \Omega_k, A_k^{cand})$ (5.5)

These equations matter because routing is where the runtime first becomes an ecology rather than a list. Cells can compete, cooperate, suppress one another, or be prioritized for downstream reasons such as cost or expected utility. This richer routing view is spelled out directly in **“The Natural Time Variable for Attractor-Based LLM and AGI Dynamics.”**

This is also why routing is the right engineering analog of basin selection. A bad runtime is not one that only produces wrong final answers. It may also be one that overcommits to a seductive local basin, suppresses a necessary rival process too early, or spreads resources too thinly across weak candidates. These failure modes are explicitly listed in **“The Natural Time Variable for Attractor-Based LLM and AGI Dynamics.”**

5.3 Local Convergence

Once a cell is active, the runtime must decide whether it has reached a usable local closure. This is local convergence.

The paper insists on a crucial distinction: local convergence is not the same as global correctness. It only means that a local semantic sub-process is stable enough to export its result. This interpretation appears directly in both **“從心跳到 LLM 韻律單元探究”** and **“The Natural Time Variable for Attractor-Based LLM and AGI Dynamics.”**

The generic convergence rule is:

$q_i(k) \geq \theta_i^{conv}$ (5.6)

where `q_i(k)` is a local convergence score. The earlier work gives a useful decomposition:

$q_i(k) = \lambda_1 \cdot align_i(k) + \lambda_2 \cdot closure_i(k) + \lambda_3 \cdot consistency_i(k) - \lambda_4 \cdot loop_i(k)$ (5.7)

This formula says that local convergence rises when outputs align, close, and remain consistent, and falls when the process fragments or loops. This exact structure appears in **“從心跳到 LLM 韻律單元探究”** and is mirrored in **“The Natural Time Variable for Attractor-Based LLM and AGI Dynamics.”**

This is the first direct mathematical link between runtime state and semantic tick boundaries. A coordination episode cannot be said to complete unless some set of active cells has crossed local closure thresholds. That principle is explicitly stated in **“The Natural Time Variable for Attractor-Based LLM and AGI Dynamics.”**

Local convergence also introduces a nontrivial engineering responsibility: a system must detect not only successful stabilization, but fragile or pathological stabilization. A cell may converge too early, loop indefinitely, stabilize with high confidence but poor transferability, or reach a false local certainty. These are not fringe pathologies; they are exactly the structured failure modes listed in the earlier runtime document.

5.4 Composition

If local convergence were the final step, the runtime would only contain isolated local minima. Higher-order reasoning appears only when locally converged outputs become ingredients of a larger semantic state. This is composition.

The simplest compositional law is:

M_(k+1) = Comp({X_out^(i)}_(i∈A_k^conv)) (5.8)

where `A_k^conv` is the subset of active cells that actually converged. The result `M_(k+1)` may be a stabilized interpretation, a new planning object, an evidence bundle, a selected branch, a summary artifact, or a message passed to another module or agent. This structure is directly described in **“The Natural Time Variable for Attractor-Based LLM and AGI Dynamics.”**

The same document also proposes a weighted composition form:

M_(k+1) = Σ_i ω_i(k)·P_i(X_out^(i)) (5.9)

where:

- `ω_i(k)` is the composition weight,
- `P_i` is a normalization or projection operator applied to the exported output of cell `i`.

This equation is only schematic, but it makes two important engineering points clear.

First, different local closures need not contribute equally. A contradiction-check artifact and a retrieval artifact may both converge, but they may deserve different weights in the next state.

Second, local outputs often cannot simply be concatenated. They may need arbitration, normalization, conflict resolution, or projection before they can coexist in a larger semantic state. This is exactly where the new paper goes beyond existing path-search views such as simple branch enumeration. **“從 LLM「傾蓋如故」和「白首如新」的差異對待 探討構造 AGI Attactor 所必須的 Heatbeat 時間本質”** explicitly says that the next step is to formalize not just path search but attractor-to-attractor composition and conflict resolution.

5.5 The Minimal Runtime Chain

The four mechanics now fit into one natural order:

1. Trigger determines which cells become relevant.
2. Routing determines which candidates become active.
3. Local convergence determines which active cells reach exportable closure.
4. Composition determines how those closures reshape the next larger state.

This can be summarized compactly as:

A_k^cand -> A_k -> A_k^conv -> M_(k+1) (5.10)

This chain is the minimal engine of episode-time runtime. It is one of the most important equations in the paper because it compresses the entire runtime grammar into a single progression. It also shows why a coordination episode is not just “some reasoning happened.” A coordination episode is a structured path through relevance, activation, local stabilization, and higher-order assembly.

At this point, the framework is no longer only a theory of time indexing. It has become a theory of runtime coordination.

5.6 Why This Section Changes the Status of the Theory

Before Sections 3–5, the proposal could still be read as a descriptive suggestion: maybe semantic episodes are a better lens than tokens. After Sections 3–5, the proposal has explicit runtime primitives, local units, and state-transition logic. It now supports concrete engineering questions:

- Which cells should be surfaced in logs?
- Which routing law best reduces brittle overcommitment?
- Which convergence proxies predict transferability?
- Which composition policies reduce conflict residue?
- At which boundaries should control be applied?

That is why these sections are the real hinge of the paper. They turn episode-time from a philosophical clock into the skeleton of a semantic runtime.

6. A Formal Episode-Level State Model

Sections 3–5 translated "attractor" language into runtime primitives. That translation now needs a mathematical skeleton. Without a structured state model, the framework remains suggestive but hard to instrument. With a state model, engineers can begin to ask disciplined questions about activation, stability, failure, and intervention. This move is developed explicitly in **"The Natural Time Variable for Attractor-Based LLM and AGI Dynamics"** and is also reflected in the outline logic of **"從心跳到 LLM 韻律單元探究."**

The central claim of this section is simple:

A coordination episode is not adequately described by one hidden vector or one output string.
It is a structured state transition over multiple coupled runtime components.

We therefore model the episode-level state as:

$S_k = (Z_k, A_k, T_k, M_k, R_k, Y_k)$ (6.1)

where:

- Z_k = latent semantic configuration,
- A_k = active cell set,
- T_k = tension vector,
- M_k = memory and retrieved context,
- R_k = routing and arbitration state,
- Y_k = current artifact set.

This equation should be read as a runtime envelope, not as a claim that every implementation must literally expose these six objects in identical form. The point is structural: higher-order reasoning is composite, and the runtime needs a composite state description.

6.1 Latent Semantic Configuration

Z_k is the component closest to what practitioners loosely call "where the system currently is" in semantic space. It may include dominant interpretations, candidate hypotheses, unresolved alternatives, compressed abstractions, or currently effective local commitments. **"The Natural Time Variable for Attractor-Based LLM and AGI Dynamics"** explicitly describes Z_k in this way and treats it as the component most closely related to semantic state-space position.

The key engineering point is that Z_k is not itself enough. Two runs may occupy similar latent interpretive territory while differing drastically in which local cells are active, which tensions dominate, or which branch commitments have already been made. That is why Z_k must be paired with the remaining runtime coordinates.

6.2 Active Cell Set

A_k is the subset of semantic cells currently active during episode k. If $C_1, C_2, \ldots, C_N$ is the full cell library, then:

$A_k \subseteq \{C_1, C_2, \ldots, C_N\}$ (6.2)

This matters because episode dynamics depend not only on the latent state, but also on which local routines are actually engaged. A retrieval cell, contradiction-check cell, branch arbitration cell, and formatting cell may all exist in the runtime library, but only some are active in a given episode. This exact role for A_k is described in **"The Natural Time Variable for Attractor-Based LLM and AGI Dynamics."**

From an engineering perspective, A_k is one of the most valuable observable proxies in the whole framework. It lets the runtime say not only "the model is thinking," but "the system is currently performing evidence gathering," or "the system

is currently resolving branch competition," or "the system is currently trying to stabilize a downstream artifact."

6.3 Tension Vector

T_k collects the major semantic tensions active during the episode:

$$T_k = (\tau_1(k), \tau_2(k), \ldots, \tau_m(k)) \quad (6.3)$$

These tensions are not decorative. They are the order-parameter-like variables that make the local dynamics path-dependent and context-sensitive. A system may be balancing recall against precision, speed against verification, brevity against justification, or novelty against consistency. The earlier runtime documents explicitly describe tensions as variables that influence both triggering and convergence.

The practical benefit of carrying T_k explicitly is that many failures become intelligible only when seen as tension failures. A branch may not fail because it lacked tokens, but because a key countervailing tension was never activated or was weighted too weakly.

6.4 Memory and Retrieved Context

M_k represents the contextual traces currently accessible to the episode. This includes retrieved knowledge, prior artifacts, conversation history, task state, and other episodic material. The earlier paper makes an important point here: at episode scale, memory is not just passive storage. It is part of live state because it changes which cells are triggerable and what counts as relevant evidence.

This is a decisive difference between ordinary chronological memory and semantic runtime memory. The question is not only "what happened earlier?" but also "what prior coordination closure is available to support the present one?" That broader runtime view is also emphasized later in **"The Natural Time Variable for Attractor-Based LLM and AGI Dynamics."**

6.5 Routing and Arbitration State

R_k stores the local control state governing competition, priority, inhibition, pending branches, and conflict resolution among cells. This may include:

- selection priorities,
- suppression rules,
- unresolved conflicts,
- phase commitments,
- and deferred alternatives.

This component is essential because without R_k, one can observe what is active but not why it was chosen over plausible rivals. In an attractor-based runtime, that omission would be fatal. Rival local basins matter not only because one of them is active, but because others were possible and were suppressed, delayed, or outvoted. That is why **"The Natural Time Variable for Attractor-Based LLM and AGI Dynamics"** explicitly treats routing and arbitration as part of state rather than as an invisible side effect.

6.6 Artifact Set

Y_k is the set of currently available exportable artifacts: summaries, branch decisions, evidence bundles, tool outputs, selected interpretations, partial plans, constraints, schema fragments, and other intermediate products. The earlier paper names Y_k as the bridge between local closures and higher-order progression.

This is one of the most practically important coordinates in the whole model. If Y_k is empty, the episode may still be exploring. If Y_k exists but is low quality, the system may have superficially stabilized. If Y_k is rich but poorly integrated, the problem may lie in composition rather than in triggering or convergence. Artifact-aware state therefore gives engineers a concrete runtime handle on semantic progress.

6.7 Episode Update Law

With the state envelope defined, the episode transition can be written as:

$$S_{k+1} = G(S_k, U_k, \Omega_k) \quad (6.4)$$

where:

- U_k is the applied control or intervention policy during episode k,

- Ω_k is the incoming observation set,
- G is the episode-level transition operator.

This equation is deliberately general. It says only that the next episode-level state is produced by the current composite state, any active intervention, and the observations accumulated during the current episode. This is enough to support later work on segmentation, monitoring, control, and benchmarking.

A useful structural interpretation, already suggested by the earlier runtime papers, is that G is not a black box in the ordinary sense. It is built from an ordered runtime logic:

relevance -> activation -> local stabilization -> export -> recomposition (6.5)

This corresponds directly to the trigger–routing–convergence–composition chain developed in Section 5. The earlier paper explicitly says the ordered runtime logic is what the state model is trying to preserve.

6.8 Deviation Dynamics and Local Stability

To discuss fragility and failure, the framework also needs a local stability form. Let S* denote a locally coherent episode-level configuration. Define deviation from that configuration as:

e_k = S_k - S* (6.6)

Near S*, local update behavior may be approximated by:

e_(k+1) = J_k e_k + η_k (6.7)

where:

- J_k = effective episode-scale Jacobian or update operator,
- η_k = perturbation due to new evidence, tool returns, memory injections, or exogenous shocks.

This is the episode-scale analog of local attractor analysis. If the relevant directions of J_k are contractive, the local coordination regime is stable. If some directions become expansive, the regime is fragile or unstable. If perturbations repeatedly push the system between rival local basins, the episode may never reach robust closure. This interpretation is stated explicitly in **“The Natural Time Variable for Attractor-Based LLM and AGI Dynamics.”**

The importance of equation (6.7) is not that engineers will directly estimate a full Jacobian in every deployment. The importance is conceptual and diagnostic: the framework can now talk rigorously about local contraction, local instability, and perturbation sensitivity at the episode scale rather than only at the token scale.

6.9 Why This State Model Matters

At this point, the framework has moved beyond metaphor. The natural time variable has been defined, the minimal runtime unit has been introduced, the core runtime mechanics have been named, and the episode-level state has been written down. This is exactly the formal turning point described in **“The Natural Time Variable for Attractor-Based LLM and AGI Dynamics.”**

The value of the state model is practical. It makes it possible to ask:

- Which tensions make certain cells trigger?
- Which routing policies increase overcommitment?
- Which memory injections destabilize a partially converged episode?
- Which artifact types support robust downstream transfer?
- Which episode-level states are locally contractive and which are fragile?

Without a model like this, semantic ticks remain evocative. With it, they become analyzable.

---

7. Completion, Fragility, and Failure Attractors

A coordination-episode framework becomes useful only when it can distinguish between different kinds of local outcome. It is not enough to say that an episode has “ended.” An episode may end because it genuinely converged, because it superficially stabilized, because it became trapped in a self-reinforcing loop, or because it failed to assemble enough structure to count as meaningful closure at all. This exact point is made centrally in **“The Natural Time Variable for Attractor-Based LLM and AGI Dynamics.”**

So the central claim of this section is:

A semantic tick is not a crude binary event.
Episode completion must be stratified by robustness, fragility, and attractor pathology.

7.1 Completion Is a Semantic Status, Not a Mere Stop Condition

In ordinary step-count frameworks, completion is often procedural. A loop ends, a function returns, a turn stops. In the semantic tick framework, this is insufficient. A coordination episode counts as complete only when local closures have produced a state that is both semantically stabilized and transferable to downstream coordination. This formulation appears almost verbatim in **"The Natural Time Variable for Attractor-Based LLM and AGI Dynamics."**

Let `Req_k` denote the subset of cells that must converge for episode `k` to count as semantically complete. Then the basic completion indicator is:

$X_k = 1$ if $Req_k \subseteq A_k^{conv}$ and $Y_{(k+1)}$ is transferable; 0 otherwise (7.1)

Equation (7.1) says that a semantic tick advances only if two conditions hold:

1. the required local sub-processes have converged, and
2. the resulting outputs are usable downstream.

This creates a deep asymmetry between token-time and semantic-time. In token-time, every token step advances the clock by construction. In semantic-time, the clock advances only when closure has semantic legitimacy.

7.2 Outcome Classes

The runtime must therefore distinguish several principal episode outcome classes. A minimal but useful vocabulary is:

state_episode(k) ∈ {COLLAPSED, COLLAPSED_BUT_FRAGILE, ATTRACTOR_LOOP, NOT_CONVERGED, ASYMMETRY_BLOCK, PENDING} (7.2)

This outcome set appears explicitly in both **"The Natural Time Variable for Attractor-Based LLM and AGI Dynamics"** and **"從心跳到 LLM 韻律單元探究."**

The states should be interpreted as follows.

COLLAPSED
The episode has reached robust local closure. Required cells converged, contradiction residue is low, loop risk is controlled, and the exported artifact is usable downstream.

COLLAPSED_BUT_FRAGILE
Closure was reached, but the stabilization is weak. The output may still be usable, yet it depends on a delicate balance that may fail under small perturbations, new evidence, or tension shifts.

ATTRACTOR_LOOP
The episode has entered a self-reinforcing but semantically unproductive regime. It repeats, rephrases, or reaffirms itself without generating meaningful new transfer value.

NOT_CONVERGED
Candidate cells may have activated, but closure conditions were not met. The episode did not stabilize enough to export a coherent result.

ASYMMETRY_BLOCK
The episode is blocked because required structural balance conditions were not satisfied. One frame, polarity, or branch dominated too early, or required counter-structures were never activated.

PENDING
There is not yet enough evidence, processing depth, or runtime development to assess meaningful closure.

This classification is not cosmetic. It expresses the central insight that semantic-time must distinguish between good basin closure and bad basin capture. A system that mistakes all local stability for success will systematically overcount fake semantic ticks. That warning is made directly in the earlier paper.

7.3 Robust Completion

A robust semantic tick is not merely one that terminated. It is one in which closure is structurally supported. The earlier paper suggests the following form:

COLLAPSED if $X_k = 1$ and $c_k \geq c^*$ and $s_k \geq s^*$ and $f_k < f^*$ (7.3)

where:

- c_k = convergence quality,
- s_k = stability quality,
- f_k = fragility pressure.

This expression makes three things explicit:

- convergence alone is not enough,
- stability matters separately,
- low fragility is required for robust closure.

That is exactly why this section is necessary. Without it, the whole framework would reduce to a completion detector. With it, the framework becomes a quality-sensitive completion model.

7.4 Fragility

Completion alone is not enough because an episode may close while remaining fragile. To capture this, the earlier runtime model defines a fragility score:

$$\varphi_k = w_1 \cdot l_k + w_2 \cdot c_k + w_3 \cdot u_k - w_4 \cdot n_k \quad (7.4)$$

where:

- l_k = loop risk,
- c_k = contradiction residue,
- u_k = unresolved tension mass,
- n_k = novelty-supported stabilization.

High ϕ_k indicates that the episode may have converged only superficially. It may be one perturbation away from basin escape, self-loop, semantic fracture, or downstream breakdown. Low ϕ_k indicates more robust local stabilization. The same fragility logic is also reflected in **“從心跳到 LLM 韻律單元探究.”**

This is one of the most practically valuable ideas in the entire paper. It means the runtime can say not only “the episode ended,” but also “the closure is brittle,” which is exactly the sort of warning a production system needs before exporting a risky artifact.

7.5 Failure Attractors

The phrase “failure attractor” is not rhetorical. It names a structured runtime pathology. A system may repeatedly fall into:

- loop-dominant states,
- contradiction-tolerant local closures,
- overcommitted branch selections,
- shallow artifact completions,
- or persistent asymmetry blocks.

These are not random mistakes. They are local regimes that can become self-reinforcing. That is why the framework treats them as attractor pathologies rather than isolated errors. This is one of the strongest reasons to adopt episode-time: many semantically important failures are not visible as single-token anomalies, but they become visible when one studies structured closure attempts across episodes.

7.6 Completion Logic for Real Systems

For engineering purposes, the completion logic can be stated more explicitly. A semantic tick should count as complete only if:

- required cells converged,
- an output artifact was produced,

- contradiction remains below threshold,
- and no critical loop-lock dominates.

This gives a practical gate for runtime advancement. It also helps explain why two systems with identical final-answer accuracy may differ radically in runtime quality. One may reach robust closure efficiently. Another may rely on fragile closure, hidden looping, or near-miss failures that barely happen to end correctly. The earlier documents explicitly argue that evaluation must therefore move beyond output accuracy alone.

7.7 Why This Section Matters

This section changes the meaning of "time" in the framework. If every local stop counted as a tick, then semantic-time would collapse back into crude event counting. By making tick advancement conditional on semantically legitimate closure and by stratifying outcomes into robust, fragile, looped, blocked, unresolved, and pending forms, the runtime preserves the real difference between token-time and episode-time. That distinction is central in **"The Natural Time Variable for Attractor-Based LLM and AGI Dynamics."**

So the practical takeaway is this:

The semantic clock does not advance because processing stopped.
It advances only when a meaningful coordination closure occurred, and the runtime must classify what kind of closure that was.

That is what makes semantic-time suitable for diagnostics, benchmarking, and later intervention control.

8. Dissipation in LLM Runtime: Drift, Breakage, and Switching Cost

A semantic runtime becomes practically valuable only when it can explain not just how episodes close, but why many of them degrade. In production systems, failure is often not a dramatic collapse. More often it appears as gradual drift, brittle formatting, unstable tool choices, repeated branch reversals, or low-quality closures that barely remain usable. This is exactly the operating regime targeted by **"Dissipative Lagrangian Decoding: Event-Triggered Short-Horizon Control for Stable, On-Task Large Language Models,"** which frames the problem as inference-time stability without retraining, and explicitly names topic drift, brittle structured outputs, erratic tool decisions, and entropy spikes as the central production failure modes.

The key move in this paper is to reinterpret those familiar operational failures at the **episode scale**. At token scale, one sees only local fluctuations. At episode scale, one sees something more informative: a local coordination process may lose coherence while still continuing to emit plausible text. A retrieval episode may start correctly and then decay into weak evidence aggregation. A branch-selection episode may never cleanly settle and instead oscillate. A formatting episode may appear complete until a latent structural inconsistency finally surfaces. In this sense, dissipation is not merely "wrong output." It is the gradual loss of semantic coordination integrity.

We therefore define an episode-scale dissipation state:

$$D_{k+1} = D_k + \xi_k - R_k \quad (8.1)$$

where:

- D_k = accumulated dissipation at episode k,
- ξ_k = newly accumulated disruption during the episode,
- R_k = repair or stabilization achieved during the episode.

Equation (8.1) is the simplest useful form. It says that every coordination episode can accumulate damage, but that some of that damage may be repaired before the episode closes. This logic is already present in **"從心跳到 LLM 韻律單元探究,"** where episode-level drift, contradiction, loop pressure, and recovery are explicitly treated as runtime observables and control targets.

A more failure-sensitive form is multiplicative:

$$D_{k+1} = (1 + r_k)D_k + a_k - R_k \quad (8.2)$$

where:

- r_k = amplification factor for existing instability,
- a_k = fresh disturbance injected during episode k.

This version matters because many real failures in advanced LLM systems are not additive. A fragile closure can make

the next episode easier to destabilize. A poor branch choice can amplify later contradiction. A bad schema repair can poison all downstream structured-output steps. The runtime therefore needs a notion of compounding semantic damage, not just instantaneous local mistakes. That same "damage plus amplification plus repair" logic is central to the heartbeat-oriented reasoning in **"從 LLM「傾蓋如故」和「白首如新」的差異對待 探討構造 AGI Attactor 所必須的 Heatbeat 時間本質"** and the engineering extension in **"從心跳到 LLM 韻律單元探究."**

To make dissipation operational, we decompose it into three primary runtime forms.

8.1 Topic and Intent Drift

The first form is semantic drift. The decoder remains fluent, but the active episode loses alignment with its intended basin. In **"Dissipative Lagrangian Decoding: Event-Triggered Short-Horizon Control for Stable, On-Task Large Language Models,"** this is operationalized by a topic-drift term built from embedding deviation against a recent semantic reference. That paper uses content-neutral, mechanism-relevant proxies such as 1 – cos(v_t, v_bar_t), where the current candidate direction is compared to an EMA of recent outputs.

At episode scale, the analogous quantity is not merely token drift but **episode-intent drift**:

drift_k = 1 - cos(E_k, G_k) (8.3)

where:

- `E_k` = current effective episode embedding or semantic summary,
- `G_k` = goal-conditioned episode reference.

This is not a claim that cosine similarity alone is sufficient. It is a practical proxy saying: if the active coordination episode is supposed to complete a particular closure, its semantic direction should remain meaningfully tied to that closure.

8.2 Structural Breakage

The second form is structural dissipation. This includes:

- broken JSON or XML structure,
- inconsistent tool-call arguments,
- unclosed code blocks,
- invalid schema states,
- or partial artifacts that cannot be safely consumed downstream.

In the decoding paper, these are grouped under format-integrity penalties and local risk checks, again using auditable, content-neutral signals rather than ideological or semantic-value classifiers. The point is not to judge beliefs, but to detect instability in structural form.

At episode scale, this becomes:

break_k = risk_fmt(Y_k, Ex_k, schema_k) (8.4)

where the system evaluates whether the current artifact set `Y_k` is compatible with the closure conditions and downstream transfer requirements of the episode.

This is crucial because many higher-order LLM systems fail not because the language model "did not know the answer," but because the artifact required for downstream coordination was structurally unfit.

8.3 Switching Cost and Mode Flapping

The third form is switching dissipation. A runtime may repeatedly shift between tool paths, branch hypotheses, planning modes, or formatting regimes without gaining meaningful closure. **"Dissipative Lagrangian Decoding: Event-Triggered Short-Horizon Control for Stable, On-Task Large Language Models"** explicitly treats unnecessary tool or route switching as a cost term and aims to make such decisions less erratic under latency constraints.

At episode scale, we write:

switch_k = cost(route_k -> route_(k+1)) (8.5)

This cost may include:

- latency penalties,

- tool invocation cost,
- context-reset cost,
- branch re-initialization cost,
- or semantic discontinuity cost.

Switching is not always bad. Sometimes a clean basin escape requires it. The runtime problem is not “avoid switching,” but “avoid wasteful flapping.” That distinction is especially important in planner–executor loops and tool-using agents.

8.4 Unified Episode Dissipation

These ingredients can be combined into a single episode dissipation functional:

$\Gamma_k = \alpha \cdot \text{drift}_k + \beta \cdot \text{break}_k + \gamma \cdot \text{switch}_k + \delta \cdot \text{loop}_k$ (8.6)

where `loop_k` represents loop-lock pressure already introduced in Section 7.

Equation (8.6) is the episode-scale counterpart of the token-level dissipation term in **“Dissipative Lagrangian Decoding: Event-Triggered Short-Horizon Control for Stable, On-Task Large Language Models.”** That earlier paper defines a local objective `L = V - λΓ`, with Γ gathering topic drift, switch costs, and format fragility into one control-relevant quantity. This paper inherits that logic and lifts it from token-local control to semantic-episode control.

The practical conclusion of this section is therefore straightforward:

Dissipation is the engineering reason episode-aware control is needed.
Without a dissipation model, the runtime can describe closure. With it, the runtime can explain why closures degrade, why fragile episodes recur, and where intervention is likely to help.

---

9. Dissipative Control at Episode Boundaries

Once dissipation has been modeled, control becomes possible. But the paper does not argue for arbitrary or constant intervention. Its claim is sharper:

Interventions should be aligned to episode boundaries and episode states, not merely to raw token arrival.

That design principle is stated explicitly in **“The Natural Time Variable for Attractor-Based LLM and AGI Dynamics”** and **“從心跳到 LLM 韻律單元探究,”** both of which argue that interventions are likely to be more effective when aligned with tick boundaries and episode classifications such as fragile closure, loop capture, or asymmetry block.

The conceptual bridge from the earlier papers is simple. The semantic runtime tells us:

- which episode is active,
- whether it is converging or degrading,
- whether closure is robust or fragile,
- and what artifact is expected next.

The dissipative-control paper then adds the missing engineering machinery: a bounded objective, short-horizon lookahead only at risky moments, and trust-region constraints so the controller remains auditable and safe. In the token-level version, that paper defines a local control law of the form $L_t = V_t - \lambda_t \cdot \Gamma_t$, combined with event-triggered short lookahead and KL / logit caps.

This paper now lifts that logic to the episode scale.

9.1 Episode-Level Intervention Policy

We define a generic episode-sensitive intervention policy:

$U_k = \text{Policy}(S_k, \text{state_episode}(k), \varphi_k)$ (9.1)

where:

- `S_k` = current episode-level state,
- `state_episode(k)` = classified episode status,

- ϕ_k = fragility score from Section 7.

This policy form appears directly in **"The Natural Time Variable for Attractor-Based LLM and AGI Dynamics"** and **"從心跳到 LLM 韻律單元探究."** Its importance is that intervention is now conditioned on structured runtime state rather than only on prompt templates or raw heuristics.

9.2 Local Objective at the Boundary

At the moment of a boundary decision, the controller evaluates not just likelihood but semantic utility and dissipation:

$$L_k(y) = V_k(y) - \lambda_k \cdot \Gamma_k(y) \quad (9.2)$$

where:

- $V_k(y)$ = episode-level value of choosing action or export y,
- $\Gamma_k(y)$ = expected semantic dissipation associated with that choice.

This is the episode analogue of the token-level Lagrangian in **"Dissipative Lagrangian Decoding: Event-Triggered Short-Horizon Control for Stable, On-Task Large Language Models."** The earlier paper applies it to tokens; here the same idea is applied to branch choices, boundary actions, artifact exports, tool escalations, or repair steps.

A short-horizon form is:

$$J(y_{k:k+H}) = \Sigma_{\tau=k}^{k+H} [V_\tau(y_\tau) - \lambda_\tau \cdot \Gamma_\tau(y_\tau)] \quad (9.3)$$

This says the controller may briefly evaluate a micro-trajectory over the next few semantic decisions rather than only the immediate local step. The key is that this lookahead should be **event-triggered**, not always-on. That is exactly the efficiency principle argued in the decoding paper, where short-horizon lookahead is activated only under entropy spikes, imminent format risk, or tool-decision boundaries.

9.3 When to Trigger Control

An episode-aware controller should not fire continuously. It should fire at semantically dangerous or semantically strategic moments. The earlier semantic-tick papers give several examples:

- inject contradiction only when local closure falsely stabilizes,
- escalate to tool use when evidence-retrieval episodes stall,
- force branch diversification when loop score crosses threshold,
- request summarization when artifact set grows but transferability drops,
- call another agent when asymmetry block persists.

Those examples can be compressed into a generic trigger set:

$$\text{trigger_ctrl}(k) = 1 \text{ if } [\varphi_k > \theta_\varphi] \text{ or } [\text{state_episode}(k) \in \text{RiskSet}] \text{ or } [\text{boundary_flag}_k = 1] \quad (9.4)$$

where RiskSet may include:

$$\{\text{COLLAPSED_BUT_FRAGILE}, \text{ATTRACTOR_LOOP}, \text{ASYMMETRY_BLOCK}, \text{NOT_CONVERGED}\} \quad (9.5)$$

The controller therefore becomes selective. It is calm during healthy semantic flow and more active when a closure is weak, a loop emerges, or a boundary choice carries high downstream cost.

9.4 Trust-Region Control

A production controller must remain bounded. That is one of the strongest engineering contributions of **"Dissipative Lagrangian Decoding: Event-Triggered Short-Horizon Control for Stable, On-Task Large Language Models."** It explicitly constrains intervention with KL bounds and logit caps so the controller cannot silently push the system far from its base behavior.

At the episode scale, the analogous trust-region constraints are:

$$KL(\pi_k^{ctrl} \,||\, \pi_k^{base}) \le \varepsilon \quad (9.6)$$

$$|\Delta \text{score}_k(y)| \le \delta \quad (9.7)$$

These equations say the boundary controller may reshape episode-level action selection, but only within bounded deviation from the base runtime distribution or baseline ranking logic.

This boundedness matters for three reasons:

1. **Auditability** — interventions are legible and measurable.
2. **Safety** — the controller cannot introduce hidden, large behavioral shifts.
3. **Deployability** — engineers can enable it gradually in shadow mode, then canary mode, then full production.

9.5 Why Boundary-Timed Control Is Stronger Than Token-Timed Control

The paper's strongest practical claim is not that token-level control is useless. It is that many of the most important interventions become more meaningful when aligned with semantic boundaries.

A token-local controller is good at smoothing immediate instability. An episode-boundary controller is good at:

- deciding whether to finalize or delay closure,
- forcing repair before export,
- escalating to tool use,
- reopening a rival branch,
- compressing or projecting artifacts,
- or deferring completion when transferability is low.

This is a different control horizon. It is not about the best next token; it is about the best next **semantic move**.

The practical thesis of this section is therefore:

Token-local control stabilizes generation.
Episode-boundary control stabilizes coordination.

A mature runtime can use both.

---

10. Reference Runtime Architecture for Episode-Time LLM Agents

The previous sections imply a concrete architecture. The paper is no longer only defining terms. It is now in a position to sketch a deployable runtime for episode-aware LLM agents. The underlying direction is already visible in **"The Natural Time Variable for Attractor-Based LLM and AGI Dynamics,"** which explicitly proposes a minimal runtime skeleton and a minimal runtime loop, and in **"從心跳到 LLM 韻律單元探究,"** which argues that runtime architecture must become episode-aware rather than step-aware only.

10.1 Design Principle

The architecture is built around one principle:

manage bounded semantic processes, not just token streams.

This means the runtime must explicitly represent:

- cells,
- episodes,
- states,
- tensions,
- artifacts,
- and policies.

That six-object semantic runtime skeleton is stated directly in **"The Natural Time Variable for Attractor-Based LLM and AGI Dynamics,"** where the minimal schema is written as:

Runtime = (Cells, Episodes, States, Tensions, Artifacts, Policies) (10.1)

This paper adopts that skeleton as its reference architecture.

10.2 Core Components

A practical episode-time LLM runtime contains at least the following components.

1. Base Decoder / Base Model Layer
The underlying LLM still performs ordinary autoregressive generation. Nothing in this paper removes or replaces that foundation.

2. Semantic Cell Library
A typed set of reusable local semantic units, each with intent, entry conditions, exit criteria, inputs, outputs, signals, and failure markers. This object design is specified in **"The Natural Time Variable for Attractor-Based LLM and AGI Dynamics"** and **"從心跳到 LLM 韻律單元探究."**

3. Episode Manager
Tracks the current coordination episode:

Episode_k = (id_k, goal_k, A_k, Req_k, T_k, Y_k, state_k, φ_k) (10.2)

This exact episode object appears in **"從心跳到 LLM 韻律單元探究."** It is the runtime object that makes semantic-time computable.

4. Trigger Scorer
Computes which cells become relevant in the current state.

5. Router / Arbitrator
Selects which candidate cells become active, including inhibition, priority overrides, and cooperative activation.

6. Convergence Detector
Estimates which active cells have reached local closure and whether their outputs are transferable.

7. Composition Layer
Combines converged local artifacts into the next episode-level state.

8. Fragility Monitor
Tracks loop pressure, contradiction residue, unresolved tension mass, and closure brittleness.

9. Dissipative Controller
Applies bounded intervention at risky boundaries using the objective from Section 9.

10. Runtime Logger / Dashboard Layer
Exposes the semantic runtime to humans and evaluation tooling.

This architecture is not speculative ornament. Most of its objects are already named in the earlier semantic-tick runtime schema, while the controller discipline and bounded deployment pattern come from **"Dissipative Lagrangian Decoding: Event-Triggered Short-Horizon Control for Stable, On-Task Large Language Models."**

10.3 Minimal Runtime Loop

A minimal episode-aware runtime loop can be written as:

detect -> trigger -> route -> monitor -> close -> project (10.3)

This loop is stated explicitly in **"從心跳到 LLM 韻律單元探究."** It differs from a standard step loop because the managed object is not "the next iteration" but a bounded semantic process.

A more expanded reference loop is:

observe -> score cells -> route active set -> run local processes -> test convergence -> classify episode state -> compose artifacts -> apply control if triggered -> start next episode (10.4)

This version makes clear how Sections 5–9 fit together as an implementation sequence.

10.4 Monitoring Layer

A semantic runtime is useful only if engineers can see it. **"The Natural Time Variable for Attractor-Based LLM and AGI Dynamics"** argues that dashboards should expose not only outputs and latency, but also:

- active cells,
- current tensions,
- convergence score,

- contradiction residue,
- novelty support,
- loop warnings,
- expected artifacts,
- current episode state,
- estimated fragility.

This paper adopts that view directly. A runtime dashboard should answer questions such as:

- Which coordination episode is active now?
- Why was it triggered?
- Which rival cells were not selected?
- Is the current closure robust or fragile?
- Is the system trapped in a bad basin?
- What artifact is supposed to emerge next?

These are the observability primitives that turn semantic-time into something engineers can debug.

10.5 Deployment Strategy

The runtime should be deployable incrementally. The control paper gives a clear pattern: **shadow mode first, then bounded enablement, then wider rollout**, with trust-region auditing and explicit logs. That logic applies equally well here.

A recommended rollout ladder is:

Phase 1 — Logging Only
Detect episode boundaries and compute cell, tension, and fragility proxies, but do not alter runtime behavior.

Phase 2 — Monitoring and Classification
Expose episode states, loop warnings, and fragile-closure indicators to engineers.

Phase 3 — Advisory Control
Run the controller in shadow mode and log what it would have done.

Phase 4 — Conservative Boundary Control
Enable bounded interventions only for a small set of high-confidence episode risks.

Phase 5 — Full Episode-Aware Orchestration
Let planners, memory systems, and tool-routing policies operate explicitly over semantic episodes.

This staged path is especially important because many organizations can adopt the observability layer well before they trust automatic runtime intervention.

10.6 Memory and Planning Implications

The architecture also changes how memory and planning should be organized. **“從心跳到 LLM 韻律單元探究”** argues that memory should be indexed not only by chronology but also by **episode role**, for example contradiction-resolution episodes, evidence-gathering episodes, branch-selection episodes, loop-recovery episodes, and artifact-compression episodes. It also argues that planning units should be represented as semantic episodes with explicit closure logic rather than as blind numbered steps.

So the reference architecture naturally supports:

retrieve(Memory, query, episode_role) -> candidate support set (10.5)

This makes the runtime better aligned with coordination rather than raw history.

10.7 Final Architectural Thesis

The architectural message of Sections 8–10 is simple:

A production-grade higher-order LLM system should not only decode tokens and call tools.

It should manage semantic episodes, measure their dissipation, classify their closure quality, and intervene at meaningful boundaries.

That is the practical engineering horizon opened by combining:

- **"The Natural Time Variable for Attractor-Based LLM and AGI Dynamics,"**
- **"從心跳到 LLM 韻律單元探究,"**
- and **"Dissipative Lagrangian Decoding: Event-Triggered Short-Horizon Control for Stable, On-Task Large Language Models."**

Send the next prompt as:

Sections 11–13

11. Instrumentation and Proxy Metrics

A semantic runtime is useful only if it can be observed. This point is made explicitly in both **"The Natural Time Variable for Attractor-Based LLM and AGI Dynamics"** and **"從心跳到 LLM 韻律單元探究,"** which argue that if semantic ticks are real, then dashboards must expose episode state rather than only outputs, tool calls, latency, and retries. Both documents say a semantic-tick-aware monitor should surface active cells, current tensions, convergence score, contradiction residue, novelty support, loop warnings, expected artifacts, current episode state, and estimated fragility.

The practical consequence is immediate: logging must move from raw chronological traces toward a structured runtime trace. In **"從心跳到 LLM 韻律單元探究,"** the proposed research roadmap makes this explicit by defining a trace stage in which logs should include tokens, tool calls, memory events, routing events, artifacts, and local metrics, so that a run can later be replayed and inspected at the level of proposed episode structure rather than only at the level of raw text emission.

The core logging object of this paper is therefore the episode trace:

$$\text{Trace}_k = (\text{episode_id}, \text{state}_k, \text{cells}_k, \text{tensions}_k, \text{artifacts}_k, \text{metrics}_k, \text{action}_k) \quad (11.1)$$

This equation is not copied from one source document verbatim, but it is the natural engineering consolidation of the schema elements proposed in **"從心跳到 LLM 韻律單元探究,"** where the episode object, state object, tension object, and artifact object are each defined separately, and the trace roadmap requires those objects to be serializable and replayable.

A minimal episode object should include:

$$\text{Episode}_k = (\text{id}_k, \text{goal}_k, A_k, \text{Req}_k, T_k, Y_k, \text{state}_k, \varphi_k) \quad (11.2)$$

This exact structure is given in **"從心跳到 LLM 韻律單元探究,"** together with a minimal field list such as episode goal, trigger source, active cells, required cells, artifacts, tension state, episode state, fragility score, completion confidence, and end marker. That document also gives the admissible episode states as `PENDING`, `ACTIVE`, `COLLAPSED`, `COLLAPSED_BUT_FRAGILE`, `ATTRACTOR_LOOP`, `NOT_CONVERGED`, and `ASYMMETRY_BLOCK`.

The state object should also remain explicit:

$$S_k = (Z_k, A_k, T_k, M_k, R_k, Y_k) \quad (11.3)$$

Again, this structure is already specified in **"從心跳到 LLM 韻律單元探究,"** along with concrete state fields such as semantic snapshot, active cell IDs, tension vector, memory references, routing state, artifact IDs, contradiction score, novelty score, loop score, and balance score. The important design choice there is that the runtime does not need perfect internal access; it only needs a stable enough proxy representation to support monitoring and control.

At the metric level, the paper recommends three classes of proxy.

11.1 Structural proxies

These track whether the semantic runtime has the shape it claims to have. Suitable structural proxies include:

- episode length in tokens and wall-clock time,
- active-cell count,
- number of routed candidates,
- required-cell completion ratio,
- artifact count and artifact type distribution,

- re-route count,
- branch fan-out,
- and boundary density per task.

These are direct engineering consequences of the schema and runtime-loop proposals in **“The Natural Time Variable for Attractor-Based LLM and AGI Dynamics”** and **“從心跳到 LLM 韻律單元探究.”** Both documents insist that traces must support episode detection, closure classification, and artifact tracking, which in turn requires these structural measurements.

11.2 Semantic-stability proxies

These track whether a local episode is converging robustly or drifting toward pathology. The most important are:

- contradiction residue,
- convergence score,
- novelty support,
- loop score,
- fragility score,
- transferability score,
- asymmetry-block indicators,
- and basin-escape success after intervention.

These metrics are explicitly named in **“The Natural Time Variable for Attractor-Based LLM and AGI Dynamics”** and **“從心跳到 LLM 韻律單元探究,”** which argue that runtime evaluation must include fragile-closure rate, loop-lock incidence, asymmetry-block frequency, transferability rate, intervention responsiveness, and similar episode-level quantities rather than only final task accuracy.

A compact derived indicator is:

quality_k = c_1·transfer_k + c_2·conv_k - c_3·contr_k - c_4·loop_k - c_5·φ_k (11.4)

The paper does not claim this exact linear form is universal. Its purpose is to make clear that “episode quality” should be computed from multiple structured proxies, not inferred from completion alone. That is the direct lesson of the earlier semantic-tick documents’ distinction between robust closure, fragile closure, looping, and blocked states.

11.3 Control and cost proxies

Because this paper integrates ideas from **“Dissipative Lagrangian Decoding: Event-Triggered Short-Horizon Control for Stable, On-Task Large Language Models,”** the runtime should also log intervention and cost signals. That paper explicitly recommends a logging schema containing `drift`, `fmt_ok`, `toolcost`, `λ`, `Δlogit`, `KL`, and trigger information, and argues that deployment quality depends on versioned controller logs and explicit diagnostics blobs.

At episode scale, the analogous control log is:

ControlLog_k = (Γ_k, λ_k, trigger_k, action_k, gain_k, cost_k, KL_k) (11.5)

where:

- `Γ_k` = dissipation estimate,
- `λ_k` = controller strength,
- `trigger_k` = reason control fired,
- `action_k` = intervention selected,
- `gain_k` = measured post-intervention benefit,
- `cost_k` = latency or coordination cost,
- `KL_k` = bounded deviation from baseline policy.

This is the natural episode-level generalization of the token-level controller schema proposed in **“Dissipative Lagrangian Decoding: Event-Triggered Short-Horizon Control for Stable, On-Task Large Language**

Models."

The practical rule for this section can therefore be compressed as follows:

A semantic runtime is instrumented well only when an engineer can answer, from logs alone:

- What episode was active?
- Why was it triggered?
- Which cells were selected and which were rejected?
- Did closure occur, and what kind?
- Was the closure fragile?
- What artifact was expected and what artifact was actually produced?
- Did intervention help, and at what cost?

Without such instrumentation, episode-time remains a vocabulary. With it, episode-time becomes an observable runtime discipline.

---

12. Experimental Program and Falsifiable Predictions

A paper of this kind is valuable only if it can fail. **"The Natural Time Variable for Attractor-Based LLM and AGI Dynamics"** states this clearly by giving explicit falsifiers, including the possibility that episode segmentation adds no explanatory power over token segmentation, that episode-based basin analysis reveals no cleaner structure or earlier warnings, that failure attractors are indistinguishable from random fluctuations, that boundary-timed interventions do not outperform arbitrary timing, and that the framework's value does not increase with coordination complexity. This section adopts that scientific posture directly.

The paper's empirical thesis can be compressed into one line already stated in that document:

The semantic tick framework is valuable only to the extent that episode-time yields stronger explanatory and intervention power than lower-level clocks for higher-order reasoning tasks.

From that thesis follow four primary predictions.

12.1 Prediction 1: Episode-indexed traces reveal cleaner geometry

Var_explained(episode-time) > Var_explained(token-time) (12.1)

This prediction is the direct content of Experimental Program I in **"The Natural Time Variable for Attractor-Based LLM and AGI Dynamics,"** which proposes collecting traces on multi-step tasks, segmenting them once by token windows and once by semantic-tick candidates, deriving comparable proxy vectors, and then comparing clustering quality, transition predictability, and local recurrence. The expected result stated there is that episode-indexed traces should show cleaner phase structure and more interpretable transition logic.

12.2 Prediction 2: Episode metrics detect failure earlier

P(failure | episode metrics at k) > P(failure | token metrics at comparable horizon) (12.2)

This is Experimental Program II in **"The Natural Time Variable for Attractor-Based LLM and AGI Dynamics."** The setup proposed there is to gather successful and failed runs, compute episode-level convergence, novelty, contradiction, and loop proxies, and compare their predictive power against token-level baselines. The expected result is earlier detection of bad attractor occupation at the episode level.

12.3 Prediction 3: Boundary-timed intervention outperforms arbitrary timing

Intervention_gain(boundary-timed) > Intervention_gain(arbitrary-timed) (12.3)

This prediction is stated explicitly in **"The Natural Time Variable for Attractor-Based LLM and AGI Dynamics,"** which proposes applying the same intervention at arbitrary token positions, wall-clock intervals, and detected tick boundaries, then comparing recovery quality or task-success gain. The expected result in that document is that boundary-timed interventions should produce stronger recovery or redirection.

12.4 Prediction 4: The value of episode-time rises with coordination complexity

Benefit(episode-time) = increasing function of coordination complexity (12.4)

This expectation is also stated directly in **“The Natural Time Variable for Attractor-Based LLM and AGI Dynamics,”** which argues that the benefit of episode indexing should grow in multi-tool, multi-step, and multi-agent settings because the mismatch between low-level clocks and meaningful coordination widens as complexity increases.

12.5 Task families

The experimental programs in the earlier documents imply a practical benchmark suite. A good suite should vary reasoning depth, contradiction load, retrieval dependence, branch ambiguity, loop susceptibility, and coordination complexity. That exact benchmark logic appears in the roadmap of **“從心跳到 LLM 韻律單元探究,”** which says that a field-level benchmark should be designed so that semantic-time advantage is measurable or falsifiable.

A minimum set of task families is:

- long-context question answering,
- strict schema or strict-format generation,
- code generation and repair,
- tool routing and tool-use sequencing,
- reflective multi-step tasks,
- and multi-agent decomposition/recomposition tasks.

This list aligns both with the episode-time evaluation agenda in **“The Natural Time Variable for Attractor-Based LLM and AGI Dynamics”** and with the task families explicitly proposed in **“Dissipative Lagrangian Decoding: Event-Triggered Short-Horizon Control for Stable, On-Task Large Language Models,”** which names long-context QA, tool routing, strict-format outputs, and creative writing as evaluation families for low-overhead control.

12.6 Measurement protocol

A disciplined measurement protocol should include the following stages.

Stage A: Trace capture
Serialize tokens, tool calls, retrievals, artifacts, routing events, and local runtime metrics. This follows the trace-instrumentation stage in **“從心跳到 LLM 韻律單元探究.”**

Stage B: Dual segmentation
Segment each trace twice: once by low-level windows and once by semantic-episode candidates. This follows Experimental Program I in **“The Natural Time Variable for Attractor-Based LLM and AGI Dynamics.”**

Stage C: Proxy extraction
Compute convergence, contradiction, novelty, loop, fragility, transferability, intervention, and cost proxies at both granularities. This is demanded jointly by the semantic-tick evaluation agenda and the logging requirements of **“Dissipative Lagrangian Decoding: Event-Triggered Short-Horizon Control for Stable, On-Task Large Language Models.”**

Stage D: Comparative analysis
Compare the two indexing schemes on description, prediction, diagnosis, intervention timing, and control gain. This “five-function” criterion is stated explicitly in **“The Natural Time Variable for Attractor-Based LLM and AGI Dynamics,”** which says a semantic-time theory should be judged by whether it improves description, prediction, diagnosis, intervention, and control.

12.7 Falsifiers

To make the paper scientifically disciplined, the following falsifiers should be stated plainly:

F1. Episode segmentation adds no explanatory power over token segmentation.
F2. Episode-based basin analysis reveals no cleaner structure or earlier warning signals.
F3. Failure-attractor signals collapse under proper controls and turn out to be noise.
F4. Boundary-timed intervention does not outperform arbitrary or token-timed intervention.
F5. Framework value does not increase with coordination complexity.

These are not rhetorical safeguards. They are the conditions under which the central pitch of the paper would be weakened or lost.

12.8 Evaluation criterion

A concise engineering criterion, already stated in **“The Natural Time Variable for Attractor-Based LLM and AGI Dynamics,”** is:

System quality = final performance + runtime semantic stability (12.5)

This line matters because it shifts evaluation away from raw answer accuracy alone. The earlier document argues that two systems with the same output accuracy may differ dramatically in loop vulnerability, fragility, intervention recoverability, unnecessary branching, artifact efficiency, and failure-attractor frequency. That is exactly the gap this paper wants to make measurable.

---

13. Deployment Path: How to Add This to Existing Agent Stacks

A framework like this will fail in practice if it demands a full rewrite. The right adoption path is incremental. This lesson comes strongly from **"Dissipative Lagrangian Decoding: Event-Triggered Short-Horizon Control for Stable, On-Task Large Language Models,"** which repeatedly emphasizes a drop-in engineering path, bounded trust-region control, versioned logging, and a deployment playbook of shadow → canary → full rollout. It also comes from **"從心跳到 LLM 韻律單元探究,"** which frames the runtime program as cumulative stages beginning with vocabulary stabilization, schema specification, trace instrumentation, and segmentation before full semantic controllers are attempted.

The deployment principle is therefore:

adopt episode-time first as observability, then as evaluation, then as bounded control, and only later as full orchestration logic.

13.1 Phase 0: Vocabulary and schema stabilization

Before intervention, teams need consistent terms. **"從心跳到 LLM 韻律單元探究"** explicitly warns that early research programs fail when terms such as episode, cell, closure, fragility, loop, and artifact are used inconsistently, and therefore proposes a stabilization stage with a compact glossary and symbol sheet. It then proposes a descriptive schema stage in which cells, episodes, tensions, artifacts, and policies are serialized in a shared format.

The practical output of Phase 0 is:

Schema_0 = serialize(Cells, Episodes, Tensions, Artifacts, Policies) (13.1)

At this stage, nothing in production behavior changes. Only the language and trace format are stabilized.

13.2 Phase 1: Logging only

The safest first deployment is observability without control. Teams add trace capture for tokens, tool calls, retrievals, routing events, artifacts, and local metrics, but they do not yet alter runtime decisions. This follows the trace-instrumentation stage proposed in **"從心跳到 LLM 韻律單元探究."**

At the same time, engineers can adopt the logging discipline from **"Dissipative Lagrangian Decoding: Event-Triggered Short-Horizon Control for Stable, On-Task Large Language Models,"** which recommends versioned controller state, explicit diagnostic blobs, and structured fields such as drift, structural validity, cost, λ, trigger, Δlogit, and KL. Even before the controller is enabled, those logging ideas give a mature template for runtime observability.

13.3 Phase 2: Episode segmentation and state classification

Once logs exist, the next phase is segmentation. This is the first major scientific bottleneck in **"從心跳到 LLM 韻律單元探究,"** which states that semantic ticks do not exist operationally until raw traces can be partitioned into bounded coordination episodes, whether by rule-based boundaries, artifact-trigger boundaries, or later learned detectors.

At the end of this phase, the system should be able to assign:

episode_state_k ∈ {PENDING, ACTIVE, COLLAPSED, COLLAPSED_BUT_FRAGILE, ATTRACTOR_LOOP, NOT_CONVERGED, ASYMMETRY_BLOCK} (13.2)

The benefit of this phase is already high even without control, because it gives teams an interpretable failure taxonomy and a much richer debugging view than step counts alone.

13.4 Phase 3: Advisory control in shadow mode

Only after segmentation and classification work reasonably well should the runtime propose interventions. At this stage the controller runs in advisory mode: it computes what it would have done, but does not yet modify live decisions. This phase is strongly aligned with the deployment guidance in **"Dissipative Lagrangian Decoding: Event-Triggered Short-Horizon Control for Stable, On-Task Large Language Models,"** which advocates bounded controllers, latency guards, and explicit logging before production enablement.

The advisory controller can already compute:

U_k^shadow = Policy(S_k, state_episode(k), φ_k) (13.3)

and compare predicted gain against actual outcomes, without taking action. This directly follows the episode-state policy form proposed in **“The Natural Time Variable for Attractor-Based LLM and AGI Dynamics”** and **“從心跳到 LLM 韻律單元探究.”**

13.5 Phase 4: Conservative boundary control

Only after shadow-mode evidence should the runtime apply bounded interventions at a narrow set of high-confidence boundaries. These might include:

- forcing repair when a strict-format artifact is about to export in fragile state,
- escalating to tool use when an evidence-gathering episode stalls,
- reopening a rival branch when loop risk exceeds threshold,
- deferring final closure when transferability remains low.

These examples are explicitly proposed in **“The Natural Time Variable for Attractor-Based LLM and AGI Dynamics”** and **“從心跳到 LLM 韻律單元探究.”**

At this phase, trust-region discipline matters. The controller should remain bounded, just as in **“Dissipative Lagrangian Decoding: Event-Triggered Short-Horizon Control for Stable, On-Task Large Language Models,”** where KL bounds and score/logit caps are required so the controller remains auditable and cannot silently introduce large behavioral shifts.

13.6 Phase 5: Full episode-aware orchestration

Only later should teams redesign planners, memory, and multi-agent coordination around episode-time. **“從心跳到 LLM 韻律單元探究”** argues that planning units should be represented as semantic episodes with explicit closure logic, and that memory should be indexed not only by chronology but also by episode role, such as contradiction-resolution, evidence-gathering, branch-selection, loop-recovery, escalation, and artifact-compression episodes.

This yields the memory-retrieval principle:

retrieve(Memory, query, episode_role) -> candidate support set (13.4)

which is stated directly in both **“The Natural Time Variable for Attractor-Based LLM and AGI Dynamics”** and **“從心跳到 LLM 韻律單元探究.”** The engineering significance is that many reasoning tasks benefit more from recalling the right kind of prior coordination closure than from recalling merely the most chronologically similar earlier chunk.

13.7 Existing stack compatibility

The framework is deliberately compatible with existing agent stacks because it does not require replacing the base model or the ordinary decoder. The closest precedent is again **“Dissipative Lagrangian Decoding: Event-Triggered Short-Horizon Control for Stable, On-Task Large Language Models,”** which explicitly positions its controller as a thin layer inserted between logits and sampler, with no base-model changes. This paper extends that philosophy upward: episode-aware runtime logic can sit on top of existing orchestration libraries as an additional monitoring and control layer rather than a total rewrite.

So the likely near-term compatibility targets are:

- tool routers,
- structured-output services,
- planner–executor loops,
- long-context summarizers,
- and multi-agent orchestration layers.

This compatibility is not speculative; it is the practical consequence of the “drop-in” and “shadow→canary→full” deployment discipline already argued in the control paper.

13.8 Final deployment thesis

The deployment message of this paper can therefore be stated very plainly:

Do not begin by trying to build a grand new AGI runtime.
Begin by making reasoning episodes visible, classifiable, and comparable.

Then add bounded, boundary-aware intervention where the evidence says it helps.

That staged path is the common ground between the research roadmap in **“從心跳到 LLM 韻律單元探究,”** the engineering implications in **“The Natural Time Variable for Attractor-Based LLM and AGI Dynamics,”** and the low-overhead deployment strategy in **“Dissipative Lagrangian Decoding: Event-Triggered Short-Horizon Control for Stable, On-Task Large Language Models.”**

14. Limits, Non-Claims, and Open Problems

A framework becomes more useful, not less, when it states clearly what it does **not** claim. This is especially important here because terms such as “episode,” “semantic runtime,” and “attractor” can easily be over-read as metaphysical commitments or as universal replacements for existing LLM analysis. The earlier documents already avoid that overreach. **“The Natural Time Variable for Attractor-Based LLM and AGI Dynamics”** explicitly presents semantic tick theory as a hypothesis to be tested by whether it improves description, prediction, diagnosis, intervention, and control, and it also lists concrete falsifiers under which the framework would lose value. **“從心跳到 LLM 韻律單元探究”** likewise frames the runtime program as a staged engineering research agenda rather than as a solved doctrine.

14.1 Non-Claim: Token-time is not being discarded

The first non-claim is foundational:

This paper does not argue that token-time is wrong. It argues that token-time is often the wrong primary clock for certain higher-order coordination questions.

At the microphysical level, autoregressive analysis remains valid:

$$x_{n+1} = F(x_n) \quad (14.1)$$

This is still the correct local language for next-token generation, hidden-state transport, and decoder-level control. The earlier paper **“The Natural Time Variable for Attractor-Based LLM and AGI Dynamics”** makes this exact distinction: token-time remains the right clock for micro-generation, while episode-time is proposed as the more natural clock for semantic coordination and higher-order runtime behavior.

So the framework is additive, not destructive:

$$\text{micro clock} + \text{semantic clock} > \text{micro clock alone} \quad (14.2)$$

The paper therefore proposes layered clocks, not a hostile replacement of one by the other.

14.2 Non-Claim: Attractors are not asserted as final ontology

The second non-claim is conceptual. This paper uses attractor language as a systems abstraction for bounded local stabilization, competition, looping, and compositional closure. It does **not** require a final proof that deployed LLM agents instantiate mathematically exact attractors in a strict dynamical-systems ontology. This restraint is already built into the earlier runtime texts, which repeatedly translate attractor language into trigger, routing, local convergence, composition, and failure-mode language rather than insisting on metaphysical certainty.

The engineering requirement is weaker and more practical:

$$\text{If a runtime abstraction improves segmentation, diagnosis, and intervention, it is useful even before full ontological closure is achieved.} \quad (14.3)$$

This is the right standard for a systems paper.

14.3 Non-Claim: Episode boundaries are not assumed perfectly sharp

A third limitation is operational. Real episode boundaries may be noisy, overlapping, or partially observer-dependent. **“從心跳到 LLM 韻律單元探究”** makes this clear by treating segmentation as one of the main research bottlenecks, and by proposing staged approaches ranging from rule-based boundaries to artifact-trigger detectors to later learned segmenters.

So this paper does not assume a magical segmentation oracle. Instead, it assumes only that approximate episode boundaries may be good enough to test whether episode-time outperforms token-time on some classes of tasks.

A useful way to state this is:

$$b_k = \text{boundary detector}(\text{trace}) + \text{noise}_k \quad (14.4)$$

The framework survives imperfect segmentation if the resulting runtime still yields better explanatory or control value

than lower-level baselines. But if segmentation quality is too poor, much of the claimed gain may disappear. That is a real open risk, not a side note.

14.4 Limitation: Proxy dependence

The runtime proposed here depends heavily on proxies:

- convergence proxies,
- contradiction proxies,
- loop proxies,
- fragility proxies,
- drift proxies,
- transferability proxies.

This is unavoidable in near-term systems. The earlier documents already rely on observable proxies rather than inaccessible "true semantic state," and the control paper **"Dissipative Lagrangian Decoding: Event-Triggered Short-Horizon Control for Stable, On-Task Large Language Models"** explicitly recommends bounded, auditable, content-light diagnostics such as drift, format validity, tool cost, controller strength, trigger status, and KL deviation.

So the framework inherits a general limitation:

proxy_k ≠ ground truth semantic state (14.5)

If a proxy is miscalibrated, the runtime may over-diagnose fragility, miss genuine basin traps, or intervene at the wrong time. This is not a flaw specific to this framework; it is a general fact of modern observability. But it must be admitted explicitly.

14.5 Limitation: Local closure does not imply global correctness

A central caution of the whole paper is that local convergence and transferability do not guarantee overall task success. An episode may robustly close and still be globally wrong because it stabilized around the wrong assumption, the wrong retrieved evidence, or the wrong branch architecture. This caution is already embedded in the distinction between `COLLAPSED` and other failure classes in **"The Natural Time Variable for Attractor-Based LLM and AGI Dynamics"** and **"從心跳到 LLM 韻律單元探究."**

In compressed form:

local stability ≠ global correctness (14.6)

The framework therefore helps with runtime quality and control, but it does not solve truth, alignment, or completeness in one stroke.

14.6 Limitation: Some tasks may not benefit much from episode-time

The episode-time advantage is predicted to increase with coordination complexity, but this implies the converse: some low-depth tasks may gain little. **"The Natural Time Variable for Attractor-Based LLM and AGI Dynamics"** explicitly frames coordination complexity as the regime where semantic ticks should become more valuable.

Therefore:

Benefit(episode-time) may be small for low-complexity tasks. (14.7)

For simple completions, one-shot classification, short pure-language continuations, or tightly constrained prompt completions, token-time plus standard decoder metrics may already be sufficient. The framework should not be oversold where it adds little.

14.7 Limitation: Runtime cost and engineering overhead

A production system that tracks cells, tensions, artifacts, boundary states, and controller logs introduces overhead. The earlier control paper repeatedly emphasizes low-overhead design and event-triggered lookahead only when needed, precisely because always-on heavy control would be too costly.

This gives a simple deployment inequality:

Gain_runtime - Cost_runtime > 0 (14.8)

If the observability and control layers cost too much latency, complexity, or maintenance effort relative to their benefit, adoption will stall. That is why the deployment path in Section 13 begins with logging and shadow mode rather than full orchestration.

14.8 Open Problem: Learning better boundary detectors

One major open problem is how to detect episode boundaries automatically and robustly. Current options include:

- rule-based segmentation,
- artifact-trigger segmentation,
- tool-event segmentation,
- and learned classifiers over traces.

The research roadmap in **“從心跳到 LLM 韻律單元探究”** already identifies segmentation as a core bottleneck.

The long-term objective can be written as:

b_k^* = argmax_b Utility(segmentation b for prediction, diagnosis, intervention) (14.9)

This is not a solved problem yet.

14.9 Open Problem: Recovering basin structure from traces

Another open problem is whether stable local basin structure can be inferred reliably from observable traces. It may be that some tasks support strong episode segmentation while others remain too entangled for meaningful basin reconstruction. The earlier paper’s falsifiers already anticipate this by allowing the possibility that episode-based basin analysis reveals no cleaner structure than lower-level views.

So a major research question is:

Can trace-level observables recover enough latent structure to support useful basin diagnostics? (14.10)

14.10 Open Problem: Controller design beyond local repair

The present paper integrates a bounded, event-triggered, short-horizon controller inspired by **“Dissipative Lagrangian Decoding: Event-Triggered Short-Horizon Control for Stable, On-Task Large Language Models.”** That is a good starting point, but it is not the end of controller design. More advanced controllers may need to manage:

- branch diversification,
- multi-agent arbitration,
- delayed closure scheduling,
- memory restructuring,
- and long-horizon coordination policies.

So another open question is:

U_k^future = richer controller(S_k, history, forecast, coordination graph) (14.11)

That controller does not yet exist in mature form.

14.11 Open Problem: Standard benchmarks for semantic runtime quality

The earlier documents make clear that standard answer-accuracy benchmarks are insufficient, but the field does not yet have a mature common benchmark for semantic runtime quality. **“從心跳到 LLM 韻律單元探究”** explicitly proposes building benchmark suites that vary contradiction load, loop susceptibility, branch ambiguity, and coordination complexity so the value of semantic-time can be measured or falsified.

A long-term benchmark objective is:

Benchmark_quality = output performance + runtime stability + intervention recoverability (14.12)

That benchmark culture still needs to be built.

14.12 Final Limit Statement

The most disciplined way to summarize the limits is this:

The framework is strongest today as a runtime hypothesis and engineering program, not as a finished universal theory.

That is not a weakness. It is exactly the right maturity level for a paper whose main contribution is to give engineers a better clock, a better runtime grammar, and a more testable control surface for higher-order LLM coordination.

---

15. Conclusion: From Decoder Steps to Semantic Runtime

This paper began with a simple question: what is the right engineering clock for higher-order LLM behavior? The answer proposed was not that token-time is false, but that token-time is often misaligned with the semantic granularity of the coordination processes engineers actually care about. For tool use, multi-step reasoning, branch arbitration, long-context synthesis, and structured artifact production, a better primary clock is often the completed coordination episode. This idea was first developed in **"Coordination-Episode Tick: The Natural Time Variable for Attractor-Based LLM and AGI Dynamics"** and extended toward runtime form in **"從心跳到 LLM 韻律單元探究."**

From that time-variable shift, the paper derived a semantic runtime. Local semantic processes were represented as semantic cells with entry conditions, exit criteria, signals, failure markers, and exportable artifacts. Episode dynamics were then expressed through four primitive operations: trigger, routing, local convergence, and composition. This gave the framework a concrete engineering grammar rather than leaving it at the level of metaphor. Those runtime primitives and state objects were already foreshadowed in the earlier semantic-tick documents, which explicitly described active cells, tension vectors, artifact sets, episode states, and fragility monitoring as first-class runtime entities.

The next step was to recognize that reasoning quality is not only about closure, but about the **quality** of closure. A semantic episode may robustly collapse, collapse but remain fragile, become trapped in a loop, fail to converge, or be blocked by structural asymmetry. This distinction matters because semantic-time would collapse back into crude event counting if every stop were treated as an equally valid tick. The runtime therefore required a fragility model, a failure-attractor taxonomy, and structured episode states. Those distinctions are explicitly laid out in both **"The Natural Time Variable for Attractor-Based LLM and AGI Dynamics"** and **"從心跳到 LLM 韻律單元探究."**

From there, the paper introduced dissipation as the engineering reason control is needed. Production failures were recast not merely as wrong answers, but as semantic drift, structural breakage, switching cost, and loop pressure. This connected the semantic runtime to the inference-time control vocabulary of **"Dissipative Lagrangian Decoding: Event-Triggered Short-Horizon Control for Stable, On-Task Large Language Models,"** which supplied the bounded, event-triggered, trust-region style of intervention. The crucial extension made here was to argue that many important interventions should be aligned to semantic episode boundaries, not merely to raw token arrival. In short:

token-local control stabilizes generation,
episode-boundary control stabilizes coordination.

The paper then showed that this framework is not only descriptive. It implies:

- a runtime schema,
- an observability discipline,
- a benchmark agenda,
- a falsifiable empirical program,
- and an incremental deployment path.

Instrumentation must expose not only outputs and latency, but also episode identities, active cells, tensions, artifacts, closure states, fragility estimates, and intervention logs. Experimental evaluation must compare episode-indexed and token-indexed traces on description, prediction, diagnosis, intervention timing, and control gain. Deployment should proceed in stages: vocabulary, schema, logging, segmentation, shadow control, conservative boundary control, and only then fuller orchestration. These are all direct extensions of the engineering and evaluation programs already proposed in the three source documents named throughout this paper.

The paper's final thesis can therefore be stated compactly:

For higher-order LLM systems, token-time remains the correct microphysical index, but episode-time is often the more natural engineering clock for semantic coordination, runtime diagnosis, and meaningful intervention. (15.1)

This is the paper's main claim. It does not solve everything. It does not prove a final ontology of reasoning. It does not eliminate the need for better segmentation, better proxies, better controllers, or better benchmarks. But it does provide something the field currently lacks: a unified engineering language that links time variable, runtime structure, failure theory, and control policy into one testable framework.

That is the transition this paper is asking the field to make:

from decoder steps to semantic runtime,
from raw progression to meaningful coordination,
and from generic prompting tricks to boundary-aware control.

Appendix A. Compact Equation Sheet

This appendix collects the backbone equations of the paper in one place for implementation reference.

A.1 Two clocks

$x_{n+1} = F(x_n)$ (A.1)

$S_{k+1} = G(S_k, \Pi_k, \Omega_k)$ (A.2)

n = micro-step index, usually token step.
k = completed coordination-episode index.

A.2 Coordination episode definition

A coordination episode is the smallest variable-duration semantic unit such that:

(i) a meaningful trigger initiates one or more local processes,
(ii) those processes interact under bounded tensions and constraints,
(iii) a local convergence condition is reached, and
(iv) a transferable output is produced. (A.3)

A.3 Minimal semantic cell

$C_i = (I_i, En_i, Ex_i, X_{in_i}, X_{out_i}, T_i, \Sigma_i, F_i)$ (A.4)

where:

- I_i = intent
- En_i = entry conditions
- Ex_i = exit criteria
- X_{in_i} = required inputs
- X_{out_i} = expected outputs
- T_i = tensions referenced by the cell
- Σ_i = observable signals
- F_i = failure markers

A.4 Episode-level state

$S_k = (Z_k, A_k, T_k, M_k, R_k, Y_k)$ (A.5)

where:

- Z_k = latent semantic configuration
- A_k = active cell set
- T_k = tension vector
- M_k = memory / retrieved context
- R_k = routing and arbitration state
- Y_k = current artifact set

A.5 Trigger, routing, convergence, composition

$a_i(k) = H_i(S_k, T_k, \Omega_k)$ (A.6)

$A_k^{cand} = \{i : a_i(k) \geq \theta_i^{trig}\}$ (A.7)

$i^* = \text{argmax}_i\, a_i(k)$ (A.8)

$A_k = \text{Route}(S_k, T_k, \Omega_k, A_k^{cand})$ (A.9)

$q_i(k) \geq \theta_i^{conv}$ (A.10)

$A_k^{conv} = \{i \in A_k : q_i(k) \geq \theta_i^{conv}\}$ (A.11)

$Y_{(k+1)} = \text{Comp}(\{X_{out}^{(i)}\}_{(i \in A_k^{conv})}, R_k, T_k)$ (A.12)

Minimal runtime chain:

$A_k^{cand} \to A_k \to A_k^{conv} \to Y_{(k+1)}$ (A.13)

A.6 Local quality and fragility

$q_i(k) = \lambda_1 \cdot \text{align}_i(k) + \lambda_2 \cdot \text{closure}_i(k) + \lambda_3 \cdot \text{consistency}_i(k) - \lambda_4 \cdot \text{loop}_i(k)$ (A.14)

$X_k = 1$ if $\text{Req}_k \subseteq A_k^{conv}$ and $Y_{(k+1)}$ is transferable; 0 otherwise (A.15)

$\varphi_k = w_1 \cdot l_k + w_2 \cdot c_k + w_3 \cdot u_k - w_4 \cdot n_k$ (A.16)

where:

- l_k = loop risk
- c_k = contradiction residue
- u_k = unresolved tension mass
- n_k = novelty-supported stabilization

Robust completion criterion:

COLLAPSED if $X_k = 1$ and $c_k \geq c^*$ and $s_k \geq s^*$ and $f_k < f^*$ (A.17)

A.7 Episode states

state_episode(k) ∈ {COLLAPSED, COLLAPSED_BUT_FRAGILE, ATTRACTOR_LOOP, NOT_CONVERGED, ASYMMETRY_BLOCK, PENDING} (A.18)

A.8 Deviation dynamics

$e_k = S_k - S^*$ (A.19)

$e_{(k+1)} = J_k\, e_k + \eta_k$ (A.20)

where:

- J_k = effective episode-scale update operator
- η_k = perturbation or exogenous shock

A.9 Dissipation

$D_{(k+1)} = D_k + \xi_k - R_k$ (A.21)

$D_{(k+1)} = (1 + r_k)D_k + a_k - R_k$ (A.22)

$\text{drift}_k = 1 - \cos(E_k, G_k)$ (A.23)

$\text{break}_k = \text{risk_fmt}(Y_k, Ex_k, \text{schema}_k)$ (A.24)

$\text{switch}_k = \text{cost}(\text{route}_k \to \text{route}_{(k+1)})$ (A.25)

$\Gamma_k = \alpha \cdot \text{drift}_k + \beta \cdot \text{break}_k + \gamma \cdot \text{switch}_k + \delta \cdot \text{loop}_k$ (A.26)

A.10 Control

$U_k = Policy(S_k, state_episode(k), \varphi_k)$ (A.27)

$L_k(y) = V_k(y) - \lambda_k \cdot \Gamma_k(y)$ (A.28)

$J(y_{k:k+H}) = \Sigma_{\tau=k}^{k+H} [V_\tau(y_\tau) - \lambda_\tau \cdot \Gamma_\tau(y_\tau)]$ (A.29)

$trigger_ctrl(k) = 1$ if $[\varphi_k > \theta_\varphi]$ or $[state_episode(k) \in RiskSet]$ or $[boundary_flag_k = 1]$ (A.30)

$RiskSet = \{COLLAPSED_BUT_FRAGILE, ATTRACTOR_LOOP, ASYMMETRY_BLOCK, NOT_CONVERGED\}$ (A.31)

$KL(\pi_k^{ctrl} \,||\, \pi_k^{base}) \leq \varepsilon$ (A.32)

$|\Delta score_k(y)| \leq \delta$ (A.33)

A.11 Instrumentation

$Trace_k = (episode_id, state_k, cells_k, tensions_k, artifacts_k, metrics_k, action_k)$ (A.34)

$Episode_k = (id_k, goal_k, A_k, Req_k, T_k, Y_k, state_k, \varphi_k)$ (A.35)

$ControlLog_k = (\Gamma_k, \lambda_k, trigger_k, action_k, gain_k, cost_k, KL_k)$ (A.36)

$quality_k = c_1 \cdot transfer_k + c_2 \cdot conv_k - c_3 \cdot contr_k - c_4 \cdot loop_k - c_5 \cdot \varphi_k$ (A.37)

A.12 Experimental predictions

$Var_explained(episode\text{-}time) > Var_explained(token\text{-}time)$ (A.38)

$P(failure \mid episode\ metrics\ at\ k) > P(failure \mid token\ metrics\ at\ comparable\ horizon)$ (A.39)

$Intervention_gain(boundary\text{-}timed) > Intervention_gain(arbitrary\text{-}timed)$ (A.40)

$Benefit(episode\text{-}time) = \text{increasing function of coordination complexity}$ (A.41)

A.13 Deployment ladder

$Schema_0 = serialize(Cells, Episodes, Tensions, Artifacts, Policies)$ (A.42)

$episode_state_k \in \{PENDING, ACTIVE, COLLAPSED, COLLAPSED_BUT_FRAGILE, ATTRACTOR_LOOP, NOT_CONVERGED, ASYMMETRY_BLOCK\}$ (A.43)

$U_k^{shadow} = Policy(S_k, state_episode(k), \varphi_k)$ (A.44)

$retrieve(Memory, query, episode_role) \rightarrow$ candidate support set (A.45)

A.14 Summary identity

$System\ quality = \text{final performance} + \text{runtime semantic stability}$ (A.46)

Appendix B. Minimal Runtime Schema

This appendix provides a compact runtime schema for implementing the framework in an operational way. Its purpose is not to prescribe one final software architecture, but to define the smallest structured object model sufficient to support:

- semantic cells,
- trigger and routing logic,
- episode-state tracking,
- convergence and fragility assessment,
- artifact transfer,
- and semantic-tick completion judgment.

This appendix consolidates the schema and runtime-object language already developed in **“The Natural Time Variable for Attractor-Based LLM and AGI Dynamics”** and **“從心跳到 LLM 韻律單元探究.”** Both documents explicitly define runtime objects for cells, episodes, states, tensions, artifacts, and policies, and they also give a minimal runtime loop and compact metric pack.

In other words, this appendix answers a practical engineering question:

What is the minimal runtime representation required to make episode-time computable?

B.1 Design Principle

A semantic runtime must represent not only what the model is outputting, but also:

- which local semantic units are currently active,
- what tensions they are negotiating,
- what outputs they are expected to produce,
- whether they are converging or looping,
- and whether the current coordination episode has reached transferable closure.

This means the runtime schema must include at least six object classes:

1. Cell
2. Episode
3. State
4. Tension
5. Artifact
6. Policy

The minimal schema can therefore be expressed as:

Runtime = (Cells, Episodes, States, Tensions, Artifacts, Policies) (B.1)

This is the smallest complete semantic-runtime skeleton proposed by the framework. It is stated explicitly in **“The Natural Time Variable for Attractor-Based LLM and AGI Dynamics.”**

B.2 Core Runtime Objects

B.2.1 Cell Object

A cell is the smallest reusable local semantic convergence unit.

Cell_i = (id_i, I_i, En_i, Ex_i, X_in^i, X_out^i, T_i, Σ_i, F_i) (B.2)

where:

- `id_i` = unique cell identifier
- `I_i` = intent
- `En_i` = entry conditions
- `Ex_i` = exit criteria
- `X_in^i` = input requirements
- `X_out^i` = output artifact types
- `T_i` = referenced tensions
- `Σ_i` = observable signals

- `F_i` = local failure markers

A minimal runtime implementation should also maintain a live execution state for each cell:

cell_status_i(k) ∈ {inactive, candidate, active, converged, fragile, looped, blocked} (B.3)

This cell definition, including both the object structure and the live status vocabulary, appears directly in **"The Natural Time Variable for Attractor-Based LLM and AGI Dynamics"** and is mirrored in **"從心跳到 LLM 韻律單元探究."**

Minimal cell fields:

- `cell_id`
- `intent`
- `entry_conditions`
- `exit_criteria`
- `required_inputs`
- `expected_outputs`
- `tension_refs`
- `signal_defs`
- `failure_defs`
- `priority`
- `phase_type`
- `parent_cell` or `parent_phase` if nested
- `retry_policy`

B.2.2 Episode Object

An episode is a bounded coordination process indexed by episode-time.

Episode_k = (id_k, goal_k, A_k, Req_k, T_k, Y_k, state_k, φ_k) (B.4)

where:

- `id_k` = episode identifier
- `goal_k` = local or macro objective
- `A_k` = active cell set
- `Req_k` = required convergent cell subset
- `T_k` = current tension state
- `Y_k` = current artifact set
- `state_k` = current episode state
- `ϕ_k` = fragility estimate

The runtime must also know whether an episode is still unfolding or already terminated:

episode_state_k ∈ {PENDING, ACTIVE, COLLAPSED, COLLAPSED_BUT_FRAGILE, ATTRACTOR_LOOP, NOT_CONVERGED, ASYMMETRY_BLOCK} (B.5)

This episode object and exact state vocabulary are already specified in **"從心跳到 LLM 韻律單元探究."**

Minimal episode fields:

- `episode_id`
- `episode_goal`

- `trigger_source`
- `start_marker`
- `active_cells`
- `required_cells`
- `artifacts`
- `tension_state`
- `episode_state`
- `fragility_score`
- `completion_confidence`
- `end_marker`

B.2.3 State Object

The runtime needs a structured episode-level state, not merely raw token history.

S_k = (Z_k, A_k, T_k, M_k, R_k, Y_k) (B.6)

where:

- `Z_k` = latent semantic configuration proxy
- `A_k` = active cells
- `T_k` = tension vector
- `M_k` = memory / retrieved context
- `R_k` = routing and arbitration state
- `Y_k` = artifact set

This state object is given directly in **“從心跳到 LLM 韻律單元探究.”**

Minimal state fields:

- `semantic_snapshot`
- `active_cell_ids`
- `tension_vector`
- `memory_refs`
- `routing_state`
- `artifact_ids`
- `contradiction_score`
- `novelty_score`
- `loop_score`
- `balance_score`

The runtime does not need perfect access to all internals. It only needs a stable enough proxy representation for state transition and monitoring purposes. That constraint is stated explicitly in **“從心跳到 LLM 韻律單元探究.”**

B.2.4 Tension Object

A tension object defines a semantic axis along which local processes are being pulled or balanced.

Tension_j = (id_j, axis_j, weight_j, threshold_j, signal_j) (B.7)

where:

- `id_j` = tension identifier
- `axis_j` = polarity pair
- `weight_j` = importance
- `threshold_j` = warning or critical threshold
- `signal_j` = observable proxy set

A runtime episode then carries a current evaluated tension vector:

$T_k = (\tau_1(k), \tau_2(k), ..., \tau_m(k))$ (B.8)

This structure and field logic are given in **“從心跳到 LLM 韻律單元探究.”**

Minimal tension fields:

- `tension_id`
- `axis = [pole_A, pole_B]`
- `description`
- `weight_global`
- `weight_by_cell`
- `warning_threshold`
- `critical_threshold`
- `signal_defs`

B.2.5 Artifact Object

Artifacts are the transferable outputs of local or global semantic closure.

Artifact_r = (id_r, type_r, source_r, payload_r, quality_r, transferable_r) (B.9)

where:

- `id_r` = artifact identifier
- `type_r` = artifact type
- `source_r` = originating cell or episode
- `payload_r` = semantic content
- `quality_r` = confidence or usability estimate
- `transferable_r` = downstream-consumable flag

This artifact definition appears in both **“The Natural Time Variable for Attractor-Based LLM and AGI Dynamics”** and **“從心跳到 LLM 韻律單元探究.”**

Typical artifact types include:

- interpretation
- evidence bundle
- contradiction report
- branch decision
- compressed summary
- plan fragment
- warning marker
- escalation request

- final answer candidate

Minimal artifact fields:

- `artifact_id`
- `artifact_type`
- `source_cell`
- `source_episode`
- `content_ref`
- `quality_score`
- `transferable`
- `downstream_targets`

Artifacts are critical because semantic ticks are defined not only by local stabilization but by exportable result production.

B.2.6 Policy Object

Policies govern trigger, routing, retry, escalation, and intervention.

Policy = (TriggerPolicy, RoutingPolicy, ConvergencePolicy, FailurePolicy, ProjectionPolicy) (B.10)

This policy bundle is stated directly in **"The Natural Time Variable for Attractor-Based LLM and AGI Dynamics."**

Minimal policy fields:

- `trigger_policy`
- `routing_policy`
- `convergence_policy`
- `failure_policy`
- `projection_policy`

B.3 Minimal Runtime Functions

The schema becomes a runtime only when object structures are paired with functions. The minimal runtime requires at least seven core functions. These signatures are given, in near-direct form, in **"The Natural Time Variable for Attractor-Based LLM and AGI Dynamics."**

B.3.1 Trigger Function

$a_i(k) = H_i(S_k, T_k, \Omega_k)$ (B.11)

Minimal signature:

```
trigger(cell_i, state_k, observation_k) -> activation_score
```

Role:

- detect semantic need
- detect contradiction pressure
- detect missing artifact demand
- detect escalation conditions

B.3.2 Candidate Set Builder

$A_k^{cand} = \{ i : a_i(k) \geq \theta_i^{act} \}$ (B.12)

Minimal signature:

```
candidate_cells(state_k) -> set[cell_id]
```

B.3.3 Routing Function

$A_k = \text{Route}(S_k, T_k, M_k, R_k, A_k^{cand})$ (B.13)

Minimal signature:

```
route(candidate_set, state_k, policy) -> active_set
```

Role:

- winner selection
- multi-cell activation
- inhibition
- dependency enforcement
- resource-aware selection

B.3.4 Convergence Evaluator

$q_i(k) \geq \theta_i^{conv}$ (B.14)

Minimal signature:

```
evaluate_convergence(cell_i, local_trace_i) -> convergence_score
```

A cell is locally complete when:

$\chi_i(k) = 1$ if $q_i(k) \geq \theta_i^{conv}$ and $X_{out}^{(i)}$ is transferable; 0 otherwise (B.15)

Minimal signature:

```
is_cell_complete(cell_i, convergence_score, artifact) -> bool
```

B.3.5 Artifact Composer

$Y_{(k+1)} = \text{Comp}(\{X_{out}^{(i)}\}_{(i \in A_k^{conv})}, R_k, T_k)$ (B.16)

Minimal signature:

```
compose(converged_artifacts, routing_state, tension_state) -> artifact_set
```

This function supports:

- merge
- select
- prioritize
- summarize
- normalize
- escalate

B.3.6 Episode Completion Function

$X_k = 1$ if $Req_k \subseteq A_k^{conv}$ and $Y_{(k+1)}$ is transferable; 0 otherwise (B.17)

Minimal signature:

```
is_episode_complete(required_cells, converged_cells, artifacts) -> bool
```

This is the core semantic-time advancement rule.

B.3.7 Episode Classifier

Minimal signature:

```
classify_episode(state_k, metrics_k) -> episode_state
```

Expected outputs:

- `COLLAPSED`
- `COLLAPSED_BUT_FRAGILE`
- `ATTRACTOR_LOOP`
- `NOT_CONVERGED`
- `ASYMMETRY_BLOCK`
- `PENDING`

B.4 Minimal Runtime Loop

The minimal semantic runtime loop is:

Observe -> Trigger -> Route -> Converge -> Compose -> Classify -> Tick (B.18)

This exact loop appears in **“The Natural Time Variable for Attractor-Based LLM and AGI Dynamics”** and **“從心跳到 LLM 韻律單元探究.”**

Its logic is:

1. Observe current state and incoming signals
2. Trigger candidate cells
3. Route an active set
4. Evaluate local convergence
5. Compose converged artifacts
6. Classify episode state
7. Advance semantic clock if the episode has meaningfully completed

This loop is the smallest operational expression of episode-time.

B.5 Minimal Schema in Structured Form

Below is a compact schema-style representation suitable for YAML, JSON, or other serialized forms. It follows the field structure explicitly proposed in the two source documents.

```
Runtime:
  cells: list
  episodes: list
  states: list
  tensions: list
  artifacts: list
  policies: list

Cell:
  cell_id: string
  intent: string
  entry_conditions: list
  exit_criteria: list
  required_inputs: list
  expected_outputs: list
  tension_refs: list
  signal_defs: list
  failure_defs: list
  priority: number
  phase_type: micro | meso | macro
  parent_cell: optional string
  retry_policy: optional object

Episode:
  episode_id: string
  episode_goal: string
```

```
  trigger_source: string
  start_marker: object
  active_cells: list
  required_cells: list
  artifacts: list
  tension_state: object
  episode_state: PENDING | ACTIVE | COLLAPSED | COLLAPSED_BUT_FRAGILE | ATTRACTOR_LOOP | NOT_CONVERGED | ASYMMETRY_BLOCK
  fragility_score: number
  completion_confidence: number
  end_marker: optional object

State:
  semantic_snapshot: object
  active_cell_ids: list
  tension_vector: object
  memory_refs: list
  routing_state: object
  artifact_ids: list
  contradiction_score: number
  novelty_score: number
  loop_score: number
  balance_score: number

Tension:
  tension_id: string
  axis: [string, string]
  description: string
  weight_global: number
  weight_by_cell: optional map
  warning_threshold: number
  critical_threshold: number
  signal_defs: list

Artifact:
  artifact_id: string
  artifact_type: string
  source_cell: string
  source_episode: string
  content_ref: object
  quality_score: number
  transferable: boolean
  downstream_targets: list

Policy:
  trigger_policy: object
  routing_policy: object
  convergence_policy: object
  failure_policy: object
  projection_policy: object
```

B.6 Minimal Episode Metrics Pack

A practical runtime should maintain a small standard metric set per episode. Both source documents propose a compact metric vector of this kind.

Metrics_k = (align_k, ΔH_k, contradiction_k, novelty_k, loop_k, balance_k, artifact_k) (B.19)

where:

- `align_k` = local alignment estimate
- `ΔH_k` = entropy drop
- `contradiction_k` = contradiction residue
- `novelty_k` = novelty support
- `loop_k` = loop tendency
- `balance_k` = symmetry / balance score
- `artifact_k` = artifact completion indicator

A minimal convergence score may then be:

c_k = α_1·align_k + α_2·ΔH_k + α_3·artifact_k - α_4·contradiction_k (B.20)

A minimal fragility score may be:

$\varphi_k = w_1 \cdot loop_k + w_2 \cdot contradiction_k + w_3 \cdot (1 - balance_k) - w_4 \cdot novelty_k$ (B.21)

This gives the runtime a compact decision layer without overcommitting to one measurement style.

B.7 Minimal Tick Advancement Logic

A semantic runtime needs a clear tick-advancement rule. The minimal version stated in the source documents is:

Advance $k \to k + 1$ if:

1. required cells converged,
2. at least one transferable artifact exists,
3. no critical loop lock dominates,
4. episode state is classifiable.

In symbolic form:

$advance_tick_k = 1$ if $X_k = 1$ and $a_k^{loop} < a^{lock}$ and $classify(E_k)$ defined; 0 otherwise (B.22)

This prevents the system from counting raw episode termination as meaningful semantic advancement.

A simpler operational form is:

$tick_k = complete(E_k)$ (B.23)

This compact tick rule is also explicitly stated in **“從心跳到 LLM 韻律單元探究.”**

B.8 Minimal Failure Handling Rules

A runtime schema is incomplete without failure handling. The source documents explicitly define minimum recovery policies for the main bad states.

NOT_CONVERGED

- retry with same cells
- relax thresholds
- escalate to evidence retrieval
- trigger summarization or decomposition

ASYMMETRY_BLOCK

- force rival-cell activation
- inject counter-frame
- widen search
- require contrast artifact

ATTRACTOR_LOOP

- break lexical / template lock
- diversify routing
- inject novelty or contradiction
- reset local episode
- escalate to macro controller

COLLAPSED_BUT_FRAGILE

- mark artifact as provisional
- require downstream verification
- preserve rival traces
- avoid irreversible commitment

A simple recovery policy may be written:

U_k = RecoveryPolicy(state_episode(k), φ_k, a_k^loop) (B.24)

This turns the schema from passive monitoring into active control.

B.9 Minimal Nested Runtime Extension

Even the minimal schema should allow for hierarchical nesting. The source documents explicitly recommend this, while also saying the full multi-scale engine need not be built immediately.

Let:

Cell_micro -> Episode_meso -> Episode_macro (B.25)

Then the basic nesting rule is:

macro_episode_K = aggregate({meso_episode_k}_(k∈K)) (B.26)

This does not require a full hierarchical controller from the start. It only requires that object IDs and parent-child references make nesting possible later.

Minimal nesting fields:

- `parent_episode_id`
- `child_episode_ids`
- `phase_type`
- `scope_level`

B.10 Practical Interpretation

This appendix can be compressed into one implementation principle:

A semantic runtime becomes computable when bounded semantic units, bounded coordination episodes, bounded artifact outputs, and bounded control policies are all serialized into one shared object model.

That is exactly why both **“The Natural Time Variable for Attractor-Based LLM and AGI Dynamics”** and **“從心跳到 LLM 韻律單元探究”** place so much emphasis on schema, trace serialization, and minimal runtime objects before attempting larger theoretical claims.

Appendix C. Worked Example: A Binary QA Task as Multi-Episode Coordination

This appendix gives a concrete worked example of the framework. Its purpose is to show that even a seemingly simple binary output such as **True** or **False** may require several distinct local semantic episodes before a stable verdict can be produced. The final binary answer is therefore not the primitive unit of reasoning. It is the folded surface output of a deeper coordination process. This exact framing is already present in **“The Natural Time Variable for Attractor-Based LLM and AGI Dynamics,”** which states that the final binary output is the folded projection of several completed semantic ticks, and in **“從心跳到 LLM 韻律單元探究,”** which presents the same appendix-level worked example and cell decomposition.

The example is intentionally modest. The goal is not to solve a hard domain task, but to make the runtime structure visible. That choice is important because the paper’s claim is not “binary tasks are hard.” The claim is that even very small observable outputs may sit on top of a much richer coordination process. This is also exactly the research agenda named in **“從 LLM「傾蓋如故」和「白首如新」的差異對待 探討構造 AGI Attactor 所必須的 Heatbeat 時間本質,”**

which says the right object of study is not simply how a final True/False appears, but how multiple sub-judgment attractors are triggered, converge, and compose into an observable output.

C.1 Example Claim

Consider the statement:

“If a model gives the same answer three times in a row, its reasoning has converged.”

We want the system to output either **True** or **False**. This is the exact example claim used in **“The Natural Time Variable for Attractor-Based LLM and AGI Dynamics.”**

At first glance, this looks like a tiny task. One might imagine that the model merely recalls a general intuition and emits a one-bit answer. But the runtime view says that a serious answer requires multiple bounded semantic episodes:

- parse the claim,
- clarify what “converged” should mean,
- test whether repetition is sufficient evidence,
- generate rival explanations,
- arbitrate between competing basins,
- and only then fold the result into a binary verdict.

This is why the example is useful. It compresses the paper’s main thesis into one visible case: the output is tiny, but the semantic runtime may not be.

C.2 Why This Example Matters

The example was chosen for three reasons.

First, it exposes the difference between **surface repetition** and **reasoning convergence**. The earlier document explicitly states that the second episode must clarify whether convergence means repeated output, process-level stability, local semantic closure, or stable behavior under perturbation, and it warns that without this clarification the system is at high risk of collapsing too early into a shallow local basin.

Second, it makes rival local basins visible. One basin says repeated answers are evidence of convergence. Another says repeated answers may arise from cached templates, shallow heuristics, repeated failure modes, low-novelty loops, or underexploration. The earlier appendix explicitly names these rival explanations.

Third, it shows why semantic ticks are not reducible to token counts. The first episode already counts as real progress not because many tokens were generated, but because a structured claim artifact was formed. The earlier appendix states this point directly: before the first closure the system only has a sentence string; after it, it has a semantically segmented claim object.

C.3 Cell Library for the Example

A minimal semantic-cell library for this task is:

- **Cell 1: Parsing Cell**
- **Cell 2: Criterion Clarification Cell**
- **Cell 3: Repetition Sufficiency Cell**
- **Cell 4: Rival Explanation Cell**
- **Cell 5: Arbitration Cell**
- **Cell 6: Verdict Fold Cell**

The later cells are named explicitly in both example appendices. In particular, **Cell 5** is defined as the arbitration cell comparing “repetition implies convergence” against rival explanations, and **Cell 6** is defined as the verdict fold cell that compresses the higher-order judgment into a binary answer. Both source documents also stress that these cells need not be literally separate modules in software; they are the minimal local semantic functions required for a serious answer.

A compact cell table is therefore:

Cell_1 = ParseClaim
Cell_2 = ClarifyConvergenceCriterion
Cell_3 = TestRepetitionAsEvidence
Cell_4 = GenerateRivalExplanations
Cell_5 = Arbitrate
Cell_6 = FoldVerdict (C.1)

The runtime does not have to instantiate these as six independently coded services. It only needs to be able to detect that these six local semantic functions are present in the trace at the episode level. That is enough for instrumentation and control.

C.4 Semantic Tick Sequence

A possible episode-level sequence is:

E_1 -> E_2 -> E_3 -> E_4 -> V (C.2)

where:

- `E_1` = parse and formalize claim
- `E_2` = clarify what counts as convergence
- `E_3` = evaluate repetition as evidence
- `E_4` = arbitrate against rival explanations
- `V` = final folded verdict

This exact sequence is given in **"從心跳到 LLM 韻律單元探究"** and the corresponding appendix of **"The Natural Time Variable for Attractor-Based LLM and AGI Dynamics."** Both documents say this sequence demonstrates the main point of the appendix: the binary answer is not the whole reasoning event; it is the projection of several completed semantic ticks.

A more explicit state progression is:

S_0 -> E_1 -> S_1 -> E_2 -> S_2 -> E_3 -> S_3 -> E_4 -> S_4 -> V -> S_5 (C.3)

where each `S_i` contains a richer artifact set and a more constrained semantic geometry than the preceding state.

C.5 Episode-by-Episode Walkthrough

C.5.1 Episode 1: Parse the Claim

The first local episode is not yet about truth. It is about semantic structure. The runtime activates a parsing cell because the raw statement contains an implication:

"If P, then Q."

Let:

P = "the model gives the same answer three times in a row"
Q = "its reasoning has converged" (C.4)

The first episode therefore exports:

Y_1 = {claim_type: implication, antecedent: P, consequent: Q} (C.5)

This exact artifact is given in the appendix of **"The Natural Time Variable for Attractor-Based LLM and AGI Dynamics."** The text there also states that this is already a meaningful semantic tick, because before this closure the system only had a sentence string, while after it the system has a structured claim object that downstream cells can operate on.

The key engineering lesson is simple: **semantic progress has occurred before any final truth judgment exists.** A token-only view may miss that and treat the whole stretch as undifferentiated "reasoning text."

A minimal episode record is:

Episode_1 = (goal = parse_claim, active_cells = {Cell_1}, artifact = Y_1, state = COLLAPSED) (C.6)

C.5.2 Episode 2: Clarify the Meaning of "Converged"

The second episode is triggered because the term **"converged"** is underspecified. The runtime detects that the claim cannot be judged until "converged" is operationalized. This trigger logic is stated directly in the worked example.

A criterion-clarification cell activates and produces candidate interpretations such as:

- output-level convergence
- hidden-process convergence
- local semantic closure
- stable reasoning under perturbation
- repeated answer under unchanged prompt

These candidate interpretations are listed explicitly in the source appendix.

A useful local artifact is:

Y_2 = {convergence_requires: process-level stability, not merely repeated output} (C.7)

This artifact also appears directly in the source appendix. The text there emphasizes that this is a crucial semantic move because it separates surface repetition from genuine reasoning convergence. It also states that without this episode, the runtime is at high risk of collapsing too early into a shallow local basin.

An important tension appears here:

surface_observability <-> deep_process_validity (C.8)

This exact tension axis is named in the source appendix. The runtime must negotiate it before the claim can be judged.

A minimal episode record is:

Episode_2 = (goal = clarify_convergence, active_cells = {Cell_2}, artifact = Y_2, tension = surface_observability <-> deep_process_validity, state = COLLAPSED) (C.9)

C.5.3 Episode 3: Test Surface Repetition as Evidence

The third episode tests the main intuitive attractor:

repeated output suggests stable reasoning (C.10)

The source appendix explicitly describes this as the "main intuitive attractor." A local cell evaluates whether repeated identical outputs provide sufficient evidence for process convergence.

A naive local closure is:

Y_3^naive = {repetition_implies_convergence: likely} (C.11)

This artifact is given directly in the source appendix.

But a stronger runtime should not stop there. The framework expects a rival-explanation cell to activate before accepting that closure. The appendix explicitly says the rival cell should generate alternatives such as:

- cached response template
- repeated shallow heuristic
- repeated failure mode
- attractor loop with low novelty
- insufficient exploration of alternatives

These rival alternatives are listed explicitly in the source appendix.

So a rival artifact is:

Y_3^rival = {repetition_has_multiple_causes: true} (C.12)

Again, this artifact appears directly in the source appendix.

At this point, two local basins are in competition:

B_repeat = “same answer indicates convergence” (C.13)

B_rival = “same answer may reflect loop or shallow stability” (C.14)

These two basin labels are also given explicitly in the source appendix, which says this is the first genuinely attractor-based moment in the example. The system is no longer merely unpacking a sentence; it is navigating between rival semantic basins.

A minimal structured record is:

Episode_3 = (goal = test_repetition, active_cells = {Cell_3, Cell_4}, artifacts = {Y_3^naive, Y_3^rival}, state = PENDING) (C.15)

The state remains `PENDING` because a true higher-order closure has not yet been reached. The competing basins are both live.

C.5.4 Episode 4: Arbitration

The fourth episode activates an arbitration cell because the local outputs are semantically incompatible. One branch says repetition is evidence of convergence; another says repetition is insufficient because several rival mechanisms can generate the same surface behavior. This exact setup is described in the source appendix.

A useful arbitration artifact is:

Y_4 = {surface_repetition_is_insufficient_for_reasoning_convergence} (C.16)

This artifact is given directly in the source appendix. The text there stresses that the artifact is stronger than a simple lexical negation because it contains the decisive semantic structure:

- the antecedent is observable,
- the consequent is deeper than the antecedent can guarantee,
- therefore the implication fails.

This is the actual high-value reasoning closure of the example. Up to this point, the system has not yet output **False**, but it has completed the reasoning structure required to do so.

A minimal arbitration record is:

Episode_4 = (goal = arbitrate, active_cells = {Cell_5}, input_artifacts = {Y_3^naive, Y_3^rival}, artifact = Y_4, state = COLLAPSED) (C.17)

C.5.5 Verdict Fold

Only now does the verdict-fold cell compress the higher-order judgment artifact into a binary answer. The source appendix explicitly says that only at this point does the final fold occur.

The folded verdict is:

Y_5 = {verdict: False} (C.18)

This is the surface-visible output, but it is not the deepest cognitive object. It is the compressed export of the preceding four episodes.

A minimal verdict record is:

Episode_5 = (goal = fold_verdict, active_cells = {Cell_6}, input_artifact = Y_4, artifact = Y_5, state = COLLAPSED) (C.19)

So the overall semantic tick chain is:

{Y_1, Y_2, Y_3^naive, Y_3^rival, Y_4} -> Y_5 (C.20)

The final binary answer is thus a low-dimensional surface fold of a richer coordination process.

C.6 Why This Is a Better Runtime View Than Token Counting

The example now makes the paper's central point visible.

If one indexed this reasoning only by tokens, one would see a stream of words and perhaps some logprob changes. But several of the most important semantic events would remain hidden:

- the claim became an implication object,
- the notion of convergence was operationalized,
- a rival explanation basin was activated,
- arbitration suppressed a seductive but shallow interpretation,
- and only then was a binary answer exported.

That is why the source appendix says the real progress in Episode 1 was not the number of tokens processed, but the formation of the structured claim artifact `Y_1`. The same reasoning applies to the later episodes: the meaningful unit is the coordination closure, not the token count.

A compact comparison is:

token-time view: one long reasoning trace -> final False
episode-time view: parse -> clarify -> test -> rivalize -> arbitrate -> fold (C.21)

This is the exact sort of gain the paper claims should generalize to other higher-order tasks.

C.7 Failure Modes in the Example

The worked example is also useful because it shows how bad local basins arise.

Early shallow collapse

The system may stop after Episode 3 with:

Y_3^naive = {repetition_implies_convergence: likely} (C.22)

and then prematurely fold:

Y_bad = {verdict: True} (C.23)

This is a textbook **COLLAPSED_BUT_FRAGILE** case. A local closure occurred, but a required rival explanation process was never allowed to mature. This kind of failure is exactly the sort of shallow basin capture the framework wants to surface.

Loop entrapment

The system may repeatedly restate:

"same answer suggests convergence"
"same answer suggests convergence"
"same answer suggests convergence"

without generating new contrast artifacts. This is an **ATTRACTOR_LOOP**. The source documents explicitly treat low-novelty repetition and loop-lock as structured failure modes rather than as mere textual redundancy.

Asymmetry block

The runtime may activate only supportive evidence cells and never the rival-explanation cell. In that case the episode is not robustly complete, even if a verdict is eventually emitted. This is an **ASYMMETRY_BLOCK** style pathology, because the needed counter-structure never entered play.

These failure interpretations align directly with the outcome vocabulary established earlier in the paper and in the source

runtime appendices.

C.8 Boundary-Timed Intervention in the Example

This example also shows where control should act.

The best intervention point is not arbitrary token number `n = 137`. It is the semantic boundary where Episode 3 has produced a seductive but insufficient local closure and Episode 4 has not yet run. That is exactly the kind of moment the framework calls a boundary-timed intervention point. The source documents explicitly propose interventions such as injecting contradiction when local closure falsely stabilizes and forcing rival-branch activation when loop or asymmetry risk rises.

A minimal boundary policy is:

$U_3 = \text{force_activate}(\text{Cell}_4)$ if $\text{state_episode}(3) = \text{COLLAPSED_BUT_FRAGILE}$ or $\varphi_3 > \theta_\varphi$ (C.24)

Another possible intervention is:

$U_3' = \text{inject_contrast_artifact}$ if $\text{loop}_3 > \theta_{\text{loop}}$ (C.25)

These are not arbitrary patches. They are exactly the sort of episode-sensitive control acts that Sections 8–10 of the paper argue for.

C.9 Compact Runtime Table

A compact runtime table for the example is:

Episode	Goal	Main active cells	Exported artifact	Likely state
E_1	Parse claim	Cell 1	`Y_1 = {claim_type, antecedent, consequent}`	COLLAPSED
E_2	Clarify convergence	Cell 2	`Y_2 = {process-level stability required}`	COLLAPSED
E_3	Test repetition / generate rivals	Cells 3, 4	`Y_3^naive`, `Y_3^rival`	PENDING
E_4	Arbitrate	Cell 5	`Y_4 = {surface repetition insufficient}`	COLLAPSED
V	Fold verdict	Cell 6	`Y_5 = {verdict: False}`	COLLAPSED

This table is simply the condensed engineering form of the source appendix walkthrough.

C.10 Appendix Summary

The worked example supports the paper's central thesis in the clearest possible way:

A final binary answer is not necessarily a primitive reasoning event. It may be the folded projection of several coordination episodes, each with its own local cells, tensions, artifacts, and possible failure modes.

That is precisely why the example appears in both **"The Natural Time Variable for Attractor-Based LLM and AGI Dynamics"** and **"從心跳到 LLM 韻律單元探究."** It makes visible, in the simplest possible setting, what the whole paper argues at larger scale: the natural engineering clock for higher-order reasoning is often not the next token, but the next completed semantic episode.

Appendix D. Benchmark and Evaluation Blueprint

This appendix turns the paper's empirical claims into a concrete benchmark program. Its purpose is not to lock the field into one dataset suite, but to define the minimum evaluation structure needed to test whether episode-time actually improves runtime understanding and control. This follows the staged roadmap in **"從心跳到 LLM 韻律單元探究,"** which explicitly asks for a benchmark suite with trace logging, episode labels or weak labels, intervention protocols, and evaluation metrics, and treats that bundle as the minimum deliverable for turning the framework into a serious empirical program.

D.1 Benchmark goal

The benchmark should answer one question:

Does episode-indexed runtime analysis provide measurable gains in explanation, prediction, diagnosis, or intervention over lower-level clocks on higher-order tasks? This is the exact research posture recommended in **“從心跳到 LLM 韻律單元探究,”** which says teams should ask whether episode-indexed runtime structure makes reasoning more measurable, diagnosable, and controllable rather than treating semantic-time as a final truth claim.

A compact benchmark objective is:

Benchmark_value = explanatory gain + actionable control gain (D.1)

D.2 Minimal benchmark package

A usable benchmark suite should include four items, matching the minimum deliverable named in **“從心跳到 LLM 韻律單元探究”**:

1. trace logging format,
2. episode labels or weak labels,
3. intervention protocols,
4. evaluation metrics.

We can write:

Benchmark_min = (Trace, EpisodeLabels, InterventionProtocol, Metrics) (D.2)

This is the smallest package that allows meaningful comparison across systems.

D.3 Task families

The benchmark should cover tasks where coordination structure matters. The control paper **“Dissipative Lagrangian Decoding: Event-Triggered Short-Horizon Control for Stable, On-Task Large Language Models”** already proposes a practical family of evaluation domains: long-context QA, tool routing, strict-format outputs, and creative writing, with metrics such as topic drift, entropy spikes, format violations, tool-use success, and overhead.

For this paper, the most relevant task families are:

- long-context QA,
- strict JSON / XML / schema generation,
- code generation and code repair,
- tool routing and tool-use sequencing,
- reflective multi-step reasoning,
- multi-agent decomposition and recomposition tasks.

A compact suite definition is:

TaskSuite = {QA_long, Format_strict, Code, ToolRoute, Reflective, MultiAgent} (D.3)

These tasks stress different parts of the semantic runtime:

- segmentation quality,
- closure quality,
- artifact transferability,
- routing stability,
- loop detection,
- and intervention timing.

D.4 Trace specification

Every benchmark task should emit a trace rich enough to support both token-time and episode-time analysis. The roadmap in **“從心跳到 LLM 韻律單元探究”** explicitly calls for logs including tokens, tool calls, memory events, routing

events, artifacts, and local metrics.

A minimum trace record is:

Trace_t = (tokens, tool_events, memory_events, routing_events, artifacts, local_metrics) (D.4)

For systems with the controller enabled, the control paper also recommends versioned diagnostic logs including drift, structural-validity status, tool cost, controller strength, score adjustments, KL, and trigger status.

So a controller-augmented trace is:

Trace_t^ctrl = Trace_t + (drift, fmt_ok, toolcost, λ, Δscore, KL, trigger) (D.5)

D.5 Dual segmentation protocol

The central comparative method is to analyze the same trace in two ways:

1. **low-level segmentation**, such as token windows, turn windows, or simple event counts,
2. **episode segmentation**, using rule-based, weak-label, or learned semantic-boundary detection.

This is exactly the comparison logic already implied by the semantic-tick evaluation program and its falsifiers.

Define:

Seg_low = segment(trace, low_level_rule) (D.6)

Seg_epi = segment(trace, episode_rule) (D.7)

The goal is not to assume `Seg_epi` is perfect, but to test whether it yields stronger structure or better control value than `Seg_low`.

D.6 Labeling strategy

A mature benchmark may eventually use gold episode labels, but the framework does not require that at the start. The roadmap in **“從心跳到 LLM 韻律單元探究”** explicitly allows weak labels as part of the minimum benchmark deliverable.

A practical labeling ladder is:

- weak boundary labels from artifact events,
- weak state labels from heuristics,
- human-verified subset labels,
- later learned label refinement.

This can be summarized as:

Labels = WeakLabels + VerifiedSubset + LearnedRefinement (D.8)

D.7 Metric groups

The benchmark should score systems in four groups.

D.7.1 Output metrics

These remain necessary:

- final-answer accuracy,
- task success,
- unit-test pass rate,
- valid schema rate,
- tool-use success.

The semantic framework does not replace output quality. It adds runtime quality beside it.

D.7.2 Runtime semantic metrics

These are the core semantic-runtime metrics already proposed in the earlier documents:

- convergence score,
- contradiction residue,
- novelty support,
- loop score,
- fragility score,
- closure success rate,
- fragile-closure rate,
- asymmetry-block rate,
- transferability rate.

A compact runtime-quality metric is:

RuntimeQuality = a·Closure - b·Fragility - c·Loop - d·Contradiction + e·Transferability (D.9)

D.7.3 Control metrics

For controller-enabled runs, use the operational metrics emphasized in **“Dissipative Lagrangian Decoding: Event-Triggered Short-Horizon Control for Stable, On-Task Large Language Models”**:

- topic drift,
- entropy spikes,
- format violations,
- tool-use stability,
- latency overhead,
- KL budget,
- trigger frequency,
- intervention gain.

A compact control-gain metric is:

ControlGain = Recovery_after_intervention - Cost_of_intervention (D.10)

D.7.4 Comparison metrics

These compare episode-time directly with lower-level clocks:

- clustering quality,
- transition predictability,
- pre-output failure signal strength,
- intervention timing gain,
- complexity-scaling benefit.

These map directly to the falsifiable predictions already stated in the semantic-tick materials:

Failure_signal_preoutput(episode metrics) > Failure_signal_preoutput(token metrics) (D.11)

Intervention_gain(boundary-timed) > Intervention_gain(arbitrary-timed) (D.12)

Benefit(episode-time) = increasing function of coordination complexity (D.13)

D.8 Success criteria

The roadmap in **“從心跳到 LLM 韻律單元探究”** states the whole program succeeds only if three conditions hold:

1. semantic episodes can be identified with useful reliability,
2. episode-time explains or predicts meaningful reasoning structure better than lower-level clocks on higher-order tasks,
3. episode-aware runtime control improves intervention, monitoring, or coordination quality.

So the benchmark-level success condition is:

Success = identifiable semantic ticks + measurable explanatory gain + actionable control gain (D.14)

D.9 Main benchmark risks

The biggest risk is benchmark gaming. **“從心跳到 LLM 韻律單元探究”** warns that benchmarks may accidentally reward artifacts of the measurement system instead of the underlying coordination logic.

A second risk comes from general benchmarking practice: systems may optimize the proxy rather than the runtime quality being measured. This same style of Goodhart risk is discussed at length in **“Purpose-Flux Belt Theory (PFBT)”**, which warns about metric gaming, window gaming, and hidden manipulations that inflate measured performance without improving true system behavior.

So the benchmark should include:

- frozen or hidden evaluation windows,
- held-out task templates,
- hidden intervention timing,
- anti-smurfing checks on episode counts,
- and audit logs for trace integrity.

We can summarize:

BenchmarkSafety = hidden windows + anti-Goodhart checks + trace integrity audits (D.15)

D.10 Minimal five-step benchmark program

For teams that want the shortest path, **“從心跳到 LLM 韻律單元探究”** already gives the compact program:

Schema -> Trace -> Segment -> Compare -> Intervene (D.16)

That is also the recommended benchmark-building order for this paper.

Appendix E. Implementation Notes for Production Systems

This appendix translates the framework into deployment guidance. Its purpose is not to specify one mandatory stack, but to show how teams can adopt the semantic-runtime and boundary-aware control ideas incrementally. The strongest engineering input here comes from **“Dissipative Lagrangian Decoding: Event-Triggered Short-Horizon Control for Stable, On-Task Large Language Models,”** especially its sections on drop-in controller design, logging schema, latency guards, versioning, and staged deployment.

E.1 Core implementation principle

Do not begin with full orchestration. Begin with observability.

This aligns with the semantic-tick roadmap’s first practical path:

Prototype_path = Schema -> Trace -> Segment -> Compare -> Intervene (E.1)

In production terms:

Deploy_path = Observe -> Classify -> ShadowControl -> ConservativeControl -> FullOrchestration (E.2)

E.2 Where the control layer lives

The control paper gives a very clear answer for token-level control: place a thin controller between logits and the sampler, with no base-model changes. It also gives concrete interface guidance: inputs include logits, top-k token IDs, top-k embeddings, rolling state, and tool/router metadata; outputs are adjusted scores and a diagnostics blob.

For this paper, the episode-aware layer should sit **above** ordinary decoding and **around** the orchestration layer:

Base model -> decoder -> orchestration runtime -> episode monitor -> boundary controller (E.3)

This means teams do not need to rewrite the model. They add:

- an episode detector,
- a semantic-state estimator,
- a fragility monitor,
- and a boundary-action policy.

E.3 Minimal production components

A production-ready minimal stack should include:

- base LLM / decoder,
- trace logger,
- episode segmenter,
- state and metric estimator,
- artifact tracker,
- fragility classifier,
- optional boundary controller,
- diagnostics store,
- rollout configuration manager.

This directly matches the semantic-runtime object model plus the engineering controller layer already described in the source papers.

A compact stack identity is:

ProdRuntime = BaseLLM + Trace + Segmenter + StateEstimator + Controller + Logs (E.4)

E.4 Logging and diagnostics

The control paper explicitly recommends:

- one-line structured logs,
- versioned controller state,
- diagnostic blobs,
- explicit trigger fields,
- KL and score-change tracking,
- and pinned controller / validator / tool-cost versions.

The minimum production log should therefore include:

- request ID,
- model version,
- runtime version,

- controller version,
- segmentation version,
- active episode ID,
- episode state,
- fragility score,
- trigger reason,
- action taken,
- latency impact,
- KL or score-shift budget use,
- output artifact IDs.

A compact diagnostic record is:

Diag_k = (runtime_ver, controller_ver, segmenter_ver, episode_id, state_k, φ_k, trigger_k, action_k, latency_k, budget_k) (E.5)

E.5 Latency guards

The control paper is emphatic that practical controllers must remain low overhead. It recommends per-step wall-clock caps, debounced triggers, and event-triggered lookahead only when risk spikes. It also says the controller should stay near-zero overhead in routine steps and only unroll short horizons when predefined triggers fire.

The episode-aware version should follow the same rule:

- no heavy episode analysis on every token,
- only evaluate boundary-sensitive logic when:
  - an artifact is about to export,
  - a tool decision boundary is near,
  - loop or fragility risk crosses threshold,
  - format breakage is imminent,
  - contradiction residue spikes.

A practical trigger rule is:

RunBoundaryControl = 1 if risk_boundary > θ or export_event = 1; else 0 (E.6)

This keeps the runtime cheap in calm regions.

E.6 Trust-region discipline

The strongest production discipline inherited from the control paper is boundedness. The controller should operate under explicit deviation caps so it remains auditable and cannot silently reshape system behavior too much. The paper recommends KL bounds and score or logit caps for exactly this reason.

At episode level, preserve the same idea:

KL(π_k^ctrl || π_k^base) ≤ ε (E.7)

|Δscore_k(y)| ≤ δ (E.8)

This should be enforced in code and surfaced in logs.

E.7 Deployment stages

The control paper recommends a deployment playbook of **shadow -> canary -> full** rollout.
The semantic-tick roadmap recommends a staged scientific program beginning with schema and trace instrumentation.

Combining both gives the recommended production ladder:

Stage 1: Schema and logging

Implement the runtime schema and trace logger only.

Stage 2: Offline segmentation and replay

Run episode detection and classification offline on saved traces.

Stage 3: Shadow boundary control

Let the controller propose actions without affecting live outputs.

Stage 4: Canary boundary control

Enable bounded control only on a small portion of traffic or narrow task slice.

Stage 5: Selective full enablement

Enable episode-aware control only where evidence shows net benefit.

This is the safest way to test the framework without a big-bang rewrite.

E.8 Versioning and configuration

The control paper explicitly recommends pinning controller version, validator version, and tool-cost version, and shipping configs as a single YAML-like object containing weights, caps, and thresholds.

The semantic-runtime layer should extend that discipline to:

- segmenter version,
- metric-pack version,
- fragility-model version,
- episode-state taxonomy version.

A compact config identity is:

Config = (controller_ver, validator_ver, segmenter_ver, metric_ver, thresholds, caps) (E.9)

Without this, benchmark comparisons and live regressions will become hard to interpret.

E.9 User-facing safeguards

The control paper also notes that strict profiles should be applied only where necessary, and that users should have an appeal or rerun path with controller disabled or relaxed if they suspect undue constraint. It also stresses cultural plurality and avoiding structure checks that accidentally encode stylistic bias.

Applied here, that means:

- use strong control only for high-risk regimes such as JSON, code, and tool routing,
- keep creative or open-ended regimes lighter,
- provide rerun modes,
- monitor fairness and creativity impacts,
- avoid treating one discourse style as the universal norm.

A practical principle is:

Control_strength = profile(task_risk, structure_requirements, user_mode) (E.10)

E.10 Memory and orchestration extensions

The roadmap in **“從心跳到 LLM 韻律單元探究”** proposes a later stage of episode-aware memory and semantic operating systems, where memory is indexed by role, topology, artifact type, and outcome state rather than chronology alone. It also proposes a modular prototype in which reasoning is episode-indexed, memory is episode-aware, interventions are boundary-sensitive, and diagnostics are attractor-aware.

So a medium-term systems extension is:

Memory_role = index({episodes_k}, by = role, topology, artifact_type, outcome_state) (E.11)

And a long-range system target is:

SemanticOS = integrate(runtime, memory, intervention, coordination, topology transfer) (E.12)

These are not first-step deployment requirements. They are later extensions once segmentation and control are already useful.

E.11 Final implementation rule

The safest production interpretation of this paper is:

use episode-time first to see, then to compare, then to advise, and only later to steer.

That rule is fully consistent with both source lines:

- the semantic-tick roadmap’s staged empirical path, and
- the control paper’s low-overhead, bounded, auditable deployment strategy.

Reference

Unified Field Theory 15: The Evolution of Exchange Bosons as Semantic Interface Structures: A Collapse-Geometric Perspective on Interaction Emergence
Unified Field Theory of Everything - Ch1~22 Appendix A~D https://osf.io/ya8tx/files/osfstorage/68ed687e6ca51f0161dc3c55

Dissipative Lagrangian Decoding_ Event-Triggered Short-Horizon Control for Stable, On-Task Large Language Models
https://osf.io/2wmky/files/osfstorage/68b45ea6b34dc4a420e4d449

SMFT AGI — From Observer-Centric Field Geometry to Production-Grade Control
https://osf.io/hj8kd/files/osfstorage/68ec02904b0f8ffc0f2e7887

Purpose-Flux Belt Theory (PFBT)
https://osf.io/yaz5u/files/osfstorage/68d01dd47195bb99223b7dfe

The Natural Time Variable for Attractor-Based LLM and AGI Dynamics
https://osf.io/hj8kd/files/osfstorage/69bdd291cb9d419aec45785b

從心跳到 LLM 韻律單元探究
https://osf.io/hj8kd/files/osfstorage/69c0383cf7d37b2e38927e73

從 LLM「傾蓋如故」和「白首如新」的差異對待 探討構造 AGI Attactor 所必須的 Heatbeat 時間本質
https://osf.io/hj8kd/files/osfstorage/69c0383b6cb8d04821185283

© 2026 Danny Yeung. All rights reserved. 版权所有 不得转载

Disclaimer

This book is the product of a collaboration between the author and OpenAI's GPT-5.4 language model. While every effort has been made to ensure accuracy, clarity, and insight, the content is generated with the assistance of artificial intelligence and may contain factual, interpretive, or mathematical errors. Readers are encouraged to approach the ideas with critical thinking and to consult primary scientific literature where appropriate.

This work is speculative, interdisciplinary, and exploratory in nature. It bridges metaphysics, physics, and organizational theory to propose a novel conceptual framework—not a definitive scientific theory. As such, it invites dialogue, challenge, and refinement.

I am merely a midwife of knowledge.